

On the Potential of Online Machine Learning in Fraud Detection and Beyond

Yixiang Wang

*Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
yixiangw@andrew.cmu.edu*

Hayden Simon

*Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
hsimon@andrew.cmu.edu*

Xianfei Yang

*Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
xianfei.y@andrew.cmu.edu*

Ozan Tonguz

*Department of Electrical and Computer Engineering
Carnegie Mellon University
Pittsburgh, USA
tonguz@andrew.cmu.edu*

Abstract—Online learning is a paradigm of machine learning wherein a model is updated incrementally upon the sequential arrival of data instances. This approach contrasts with conventional batch learning, which presupposes the availability of the entire dataset for training a priori. By processing data points sequentially, online algorithms facilitate continuous model adaptation, rendering them inherently suitable for applications characterized by large-scale data streams and non-stationary distributions, also known as concept drift. The capacity of online learning to operate under limited memory and latency constraints has driven its adoption in numerous domains, including personalized recommendation, financial market prediction, and fraud detection. This review provides a comprehensive survey of foundational online learning algorithms, organized according to a structured taxonomy that encompasses online supervised learning, online unsupervised learning, and other categories. A primary contribution of this work is a critical analysis of the practical application of these algorithms, with a specific focus on their deployment in fraud detection. Through an examination of the strengths and limitations of online learning in operational environments, this review delineates the gap between theories and practical performance. Finally, we identify and discuss promising directions for future research aimed at bridging this divide for the machine learning community.

Index Terms—Online learning, Convex Optimization, Fraud Detection

I. INTRODUCTION

The volume, velocity, and variety of data generated by modern digital systems have reached a scale at which traditional batch machine learning pipelines cannot react quickly enough to preserve accuracy or operational relevance and facing high cost when it needs retraining on the new data. High-frequency financial transactions, streaming sensor feeds, and real-time user interactions all demand predictive models that can adapt within milliseconds, not hours or days. Online machine learning (OML) addresses this demand by updating model parameters incrementally as each new example arrives, thereby sustaining performance in non-stationary environments while operating under strict latency and memory budgets.

This paradigm offers three essential advantages over offline training: (i) temporal adaptivity to concept drift; (ii) bounded storage, because historical data need not be retained; and (iii) low computational overhead, owing to single-pass updates. These properties render OML particularly suitable for domains in which data distributions evolve, labels may be delayed, and operational constraints preclude retraining from scratch.

Among the many application areas that benefit from OML, fraud detection is exemplary. Payment networks process millions of transactions per hour, each presenting a potential fraud vector whose statistical signature can shift as adversaries continuously refine their tactics. Rapid adaptation is therefore necessary. Additional domains, including recommender systems, cybersecurity monitoring, industrial IoT, and healthcare systems, face similar situations and can profit from the principled use of OML algorithms.

The remainder of this paper is organized as follows. Section 2 introduces the online learning framework and defines key concepts. Section 3 presents a comprehensive overview of fundamental online learning algorithms, spanning supervised, unsupervised, and deep learning approaches. Section 4 explores real-world applications, with a particular focus on fraud detection. Finally, Section 5 identifies open challenges and highlights promising directions for future research.

II. BACKGROUND & DEFINITION

A. Framework

Online learning is a sequential prediction framework in which, at each time step t , a learner observes an input x_t , issues a prediction $\hat{y}_t$, receives the ground-truth label y_t , suffers a loss, and immediately adjusts its parameters or internal state before the next instance arrives. The framework of online learning can be represented as follows:

Algorithm 1 Online Learning

```
for  $t = 1, 2, \dots$  do
  Receive question  $\mathbf{x}_t \in \mathcal{X}$ 
  Predict  $\hat{y}_t \in \mathcal{D}$ 
  Receive true answer  $y_t \in \mathcal{Y}$ 
  Suffer loss  $\ell(\hat{y}_t, y_t)$ 
  Update parameter  $\mathbf{w}$ 
end for
```

B. Regret

One key metric measuring OML algorithm performance is *Regret*, which is the cumulative difference between the loss incurred by the online algorithm and the loss of the best fixed decision in hindsight. Formally, for a sequence of convex loss functions $f_1, f_2, \dots, f_T$ and decisions $\mathbf{w}_1, \dots, \mathbf{w}_T$ made by the algorithm, the regret after T rounds is defined as:

$$\text{Regret}(T) = \sum_{t=1}^T f_t(\mathbf{w}_t) - \min_{\mathbf{w} \in \mathcal{S}} \sum_{t=1}^T f_t(\mathbf{w}),$$

where $\mathcal{S}$ is the feasible decision set. The goal of an online algorithm is to achieve sublinear regret, i.e., $\text{Regret}(T) = O(T)$, which ensures that the average regret converges to zero as $T \rightarrow \infty$.

C. Online Convex Optimization

Online convex optimization (OCO) provides the theoretical backbone for many online learning algorithms. The standard OCO framework involves a sequential game between a learner and an environment over T rounds. At each round, the learner selects a decision vector $\mathbf{w}$ from a non-empty, closed, and convex set $\mathcal{S} \in \mathbb{R}^d$. Subsequently, a convex loss function $f_t : \mathcal{S} \rightarrow \mathbb{R}$ is revealed, and the learner incurs the loss $f_t(\mathbf{w}_t)$. The primary objective is to minimize the regret as defined above.

Many online supervised learning problems can be formulated as specific instantiations of OCO. For instance, in regularized online linear classification, the loss function at each round is

$$f_t = \ell(y_t, \langle \mathbf{w}, \mathbf{x}_t \rangle) + \lambda \|\mathbf{w}\|_2^2$$

Here, $(\mathbf{x}_t, y_t)$ represents the feature-label pair, $\ell(\cdot, \cdot)$ is a convex loss function and the second term is an L_2 -regularized with weight $\lambda > 0$. Applying standard optimization schemes to this objective function yields well-known algorithms such as Online Gradient Descent and Online Mirror Descent. Furthermore, advanced second-order methods, such as the Adaptive Regularization of Weights (AROW) algorithm, can be interpreted as mirror descent updates employing time-varying Bregman divergences that adapt to the geometry of the data. Similarly, budgeted kernelized perceptron variants are equivalent to projected gradient methods within a Reproducing Kernel Hilbert Space (RKHS), subject to constraints on the number of support vectors.

Most importantly, OCO delivers sub-linear regret guarantees (e.g., $O(\sqrt{T})$) for strongly convex losses, which translate directly into mistake bounds for online prediction. Moreover, the

framework naturally accommodates adaptive learning rates, regularization for sparsity or stability, and bandit feedback via gradient-estimation techniques, thereby unifying seemingly disparate online-learning algorithms under a single convex-analytic lens.

A key strength of the OCO is performance guarantees. For general convex loss functions, it is possible to achieve sub-linear regret on the order $O(\sqrt{T})$. If the loss functions are additionally strongly convex, the regret bound can be improved $O(\log T)$. These regret bounds can be directly translated into mistake bounds for online classification and prediction tasks. The framework is also sufficiently general to accommodate extensions such as adaptive learning rates, various forms of regularization, and bandit feedback settings through gradient estimation. This versatility allows for the unification of seemingly disparate online algorithms under a single, coherent analytical perspective.

D. Taxonomy

The following section organizes online learning algorithms according to a two-dimensional taxonomy defined by: (1) the feedback availability and (2) the architectural class of the model (i.e., deep versus non-deep). This structure is motivated by two principal factors. First, the availability of feedback ranging from complete labels to partial or entirely absent signals fundamentally determines an algorithm's update rule, theoretical guarantees, and application scenarios. Second, the emergence of deep learning merits a distinct classification. Deep models introduce unique computational profiles, enhanced representational capacity, and significant challenges related to continual learning (e.g., catastrophic forgetting) that diverge markedly from those of classical linear or kernel-based methods.

- 1) **Online Supervised Learning:** This is most common algorithm where the feedback is fully available that the learner can update its model in a single pass at each round. Classical examples include the Perceptron, Online Gradient Descent, Passive-Aggressive algorithms, and their second-order or kernelized variants. Performance is typically analysed via regret or mistake bounds with respect to the best fixed predictor in hindsight.
- 2) **Online Semi-supervised Learning:** This paradigm addresses scenarios where only a fraction of instances are labeled, label acquisition is delayed, or supervisory signals are weak (e.g., pairwise constraints). Algorithms in this category must leverage the unlabeled data stream to augment the limited labeled data. Common techniques include online label propagation, confidence-based self-training, and self-supervised pretext tasks, all while carefully managing the risk of confirmation bias.
- 3) **Online Unsupervised Learning:** No ground-truth labels are provided. The objective is to uncover structure or detect deviations in the data stream in real time. Representative tasks include streaming clustering, density-ratio estimation, and anomaly detection via Online Isolation Forests or adaptive autoencoders. Evaluation focuses

on reconstruction loss, cluster stability, or time-lagged detection accuracy rather than conventional classification metrics.

- 4) **Online Deep Learning:** This category comprises algorithms (supervised, semi-supervised, or unsupervised) that employ deep neural architectures and update them incrementally. Examples range from truncated back-propagation through time for sequence models, to rehearsal based continual learning in vision transformers, to graph neural networks that evolve with streaming edges. Key issues include catastrophic forgetting, memory efficient replay, and maintaining real-time inference latency despite large parameter counts.

III. A TAXONOMY OF ONLINE LEARNING

A. Online Supervised Learning

1) **First-Order Learning:** First-order online learning refers to a family of algorithms in which the model is updated solely based on first-order information, such as the gradient or subgradient of the loss function. In this section, we review several fundamental first-order algorithms, including Perceptron, Winnow, Online Gradient Descent, and Passive-Aggressive methods.

Perceptron

The Perceptron algorithm is a foundational online algorithm for binary classification that learns a linear separating hyperplane. At each iteration, the algorithm uses hinge loss function $L_t(\mathbf{w}) = \max(0, 1 - y_t(\mathbf{w} \cdot \mathbf{x}_t))$. It receives an instance $\mathbf{x}_t$, makes a prediction $\hat{y}_t$ using its current weight vector $\mathbf{w}_t$, receives true label y_t , and upon misclassification $y_t(\mathbf{w}_t \cdot \mathbf{x}_t) \leq 0$, updates the weight vector by adding or subtracting the instance's feature vector $\mathbf{w}_{t+1} = \mathbf{w}_t + y_t \mathbf{x}_t$.

Algorithm 2 Perceptron

```

initialize:  $\mathbf{w}_1 = 0$ 
for  $t = 1, 2, \dots, T$  do
  Receive instance  $\mathbf{x}_t$ 
  Predict  $\hat{y}_t = f_t(\mathbf{x}_t) = \text{sign}(\mathbf{w}_t \cdot \mathbf{x}_t)$ ;
  Receive the true class label  $y_t \in \{+1, -1\}$ ;
  if  $\hat{y}_t \neq y_t$  then
     $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_t \mathbf{x}_t$ ;
  else
     $\mathbf{w}_{t+1} = \mathbf{w}_t$ 
  end if
end for

```

If there exists a unit vector $\mathbf{u}(\|\mathbf{u}\|_2 = 1)$ such that all examples are correctly classified with a margin $\gamma > 0$ and $|\mathbf{x}_t| \leq R$ for all t , the mistake bound is $M \leq (\frac{R}{\gamma})^2$, which is independent of the instances number and dependent only on the margin and norm of inputs.

Winnow Winnow is a multiplicative-weight-update approach introduced to deal with high-dimensional spaces and noisy data, which are limitations of the Perceptron algorithm. It originally was designed to learn classes of Boolean functions characterized as k -monotone disjunctions.

Algorithm 3 Winnow

```

Initialize:  $\mathbf{w}_1 = \mathbf{1} \in \mathbb{R}_+^d$ , threshold  $\theta > 0$ , promotion factor  $\alpha > 1$ 
for  $t = 1$  to  $T$  do
  Receive instance  $\mathbf{x}_t \in \{0, 1\}^d$ 
  predict  $\hat{y}_t = \mathbb{I}[\langle \mathbf{w}_t, \mathbf{x}_t \rangle \geq \theta]$ 
  Receive label  $y_t \in \{0, 1\}$ 
  if  $\hat{y}_t \neq y_t$  then
    for  $i$  with  $\mathbf{x}_t[i] = 1$  do
       $\mathbf{w}_{t+1}[i] \leftarrow \mathbf{w}_t[i] \cdot \alpha^{(2y_t - 1)}$ 
    end for
  else
     $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t$ 
  end if
end for

```

Only active features are updated (e.g. $\mathbf{x}_t[i] = 1$). Specifically, in the case of a false negative ($y_t = 1$), the weight is promoted by a factor $\mathbf{w}_{t+1}[i] = \mathbf{w}_t[i] \cdot \alpha$. In contrast, for a false positive ($y_t = 0$), the weight is demoted $\mathbf{w}_{t+1}[i] = \mathbf{w}_t[i] / \alpha$. If there are at most k relevant features out of d total, and the target function is a monotone disjunction. Then Winnow makes at most $M \leq O(k \log d)$ mistakes, which is logarithmic in the number of features and linear in the number of relevant ones, making it highly effective for sparse feature settings. Several variants have been proposed such as Winnow2, which uses a symmetric promotion and demotion factor, Balanced Winnow, and Winnow with Margin.

Online Gradient Descent (OGD) Online Gradient Descent (OGD) was proposed to serve as a general-purpose frameworks for online optimization that are principled, uses convexity, and accommodate a wide class of loss functions beyond standard classification settings. Unlike specialized algorithms such as Perceptron and Winnow, which are limited to particular tasks (e.g., binary classification with 0-1 loss or monotone disjunction learning), OGD is broadly applicable to any arbitrary convex loss function in an online learning scenario.

Algorithm 4 Online Gradient Descent (OGD)

```

Input: Learning rate  $\eta > 0$ , convex domain  $\mathcal{S}$ 
Initialize:  $\mathbf{w}_1 \in \mathcal{S}$ 
for  $t = 1$  to  $T$  do
  Receive instance  $\mathbf{x}_t \in \mathbb{R}^d$ 
  Predict  $\hat{y}_t$  using  $\mathbf{w}_1$ 
  Observe convex loss function  $f_t : \mathcal{D} \rightarrow \mathbb{R}$ 
  Compute subgradient  $g_t \in \partial f_t(\mathbf{w}_t)$ 
  Update:  $\mathbf{w}_{t+1} \leftarrow \Pi_{\mathcal{S}}(\mathbf{w}_t - \eta g_t)$ 
end for

```

Building on Online Gradient Descent (OGD), the Online Mirror Descent (OMD) algorithm extends the Online Convex Optimization (OCO) framework by performing updates in a dual space rather than the primal, enabling greater flexibility in constrained or non-Euclidean domains. OGD is therefore a

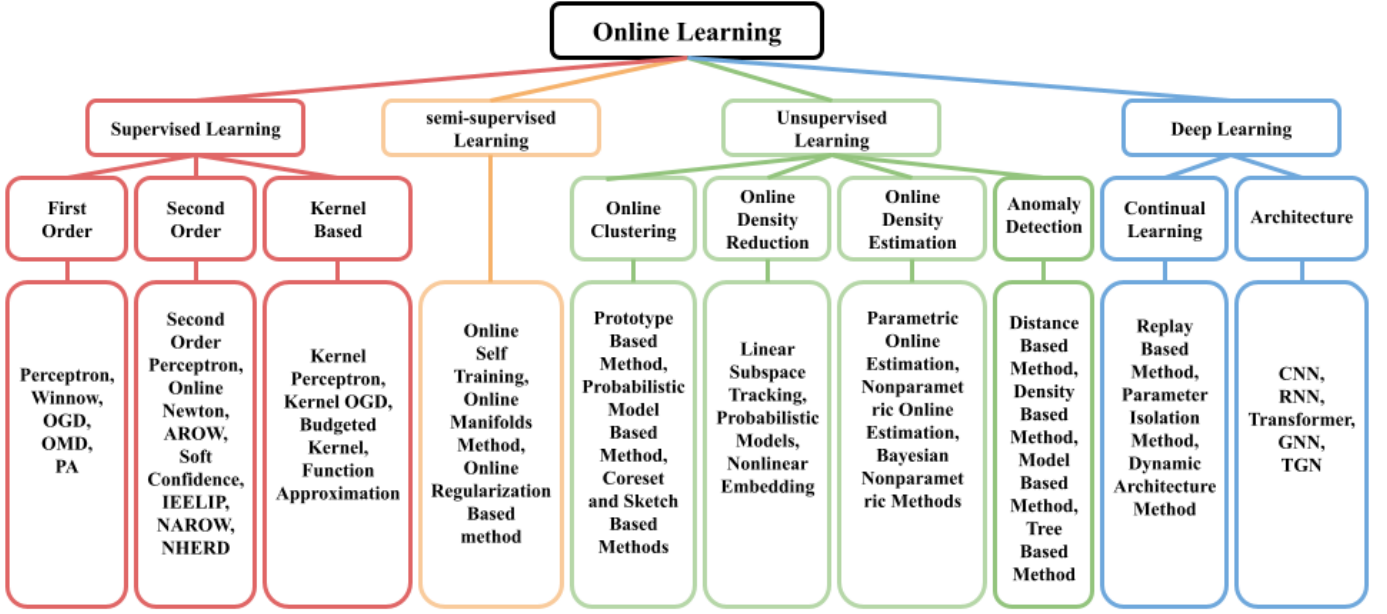


Fig. 1. Taxonomy of online learning methods across supervised, semi-supervised, unsupervised, and deep learning categories.

special case of OMD when the regularization function is the squared Euclidean norm, $\psi(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|_2^2$. The generality of OMD subsumes a wide range of online algorithms—including Exponentiated Gradient (EG), Online Newton Step (ONS), Matrix Exponentiated Gradient (MEG), Tsallis entropy-based mirror descent, OGD with lazy projections, and the p -norm algorithm—each defined by specific choices of regularization that induce different geometries on the parameter space.

Passive-Aggressive Another widely studied category of online algorithms is the Passive-Aggressive (PA) family, which primarily focuses on margin-based classification tasks. The core principle of PA, as the name suggests, is to update the model’s weight vector only when the margin constraint is violated $y_t \langle \mathbf{w}_t, \mathbf{x}_t \rangle \leq 1$ (Passive), and to do so just enough to correct the current mistake while minimally deviating from the existing model (Aggressive). This principle is formalized as following constraint optimization problem:

$$\mathbf{w}_{t+1} = \arg \min_{\mathbf{w} \in \mathbb{R}^d} \|\mathbf{w} - \mathbf{w}_t\|^2 \quad \text{s.t. } \ell_t(\mathbf{w}) = 0$$

with a closed form solution:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \tau y_t \mathbf{x}_t \quad \text{where } \tau_t = \frac{\ell_t}{\|\mathbf{x}_t\|^2}$$

To address non-separable and noisy data, soft-margin variants of the Passive-Aggressive (PA) algorithm introduce a regularization parameter $C > 0$ to balance margin satisfaction and update magnitude. The two main variants are **PA-I**, which adds a linear penalty on the slack variable ξ_t , and

PA-II, which applies a squared penalty to encourage smaller updates. In both cases, the slack variable satisfies the constraint $y_t \langle \mathbf{w}, \mathbf{x}_t \rangle \geq 1 - \xi_t$. Each variant defines a specific optimization problem and admits a corresponding closed-form update.

PA-I with Linear Slack Penalty:

$$\mathbf{w}_{t+1} = \arg \min \frac{1}{2} \|\mathbf{w} - \mathbf{w}_t\|^2 + C \cdot \xi_t \quad (1)$$

$$\tau_t = \min \left\{ C, \frac{\ell_t}{\|\mathbf{x}_t\|^2} \right\} \quad (2)$$

PA-II (soft-margin with squared slack):

$$\mathbf{w}_{t+1} = \arg \min \frac{1}{2} \|\mathbf{w} - \mathbf{w}_t\|^2 + C \cdot \xi_t^2 \quad (3)$$

$$\tau_t = \frac{\ell_t}{\|\mathbf{x}_t\|^2 + \frac{1}{2C}} \quad (4)$$

Other First-Order Algorithms

2) **Second-Order Online Learning:** Second-order online learning algorithms leverage second-order information, such as curvature or variance estimates, to adaptively scale the update direction for each feature. It often achieves faster convergence and better performance in high-dimensional and sparse spaces than first-order algorithms.

Second-Order Perceptron In contrast to the original Perceptron algorithm, the Second-Order Perceptron incorporates

accumulated historical information by maintaining a covariance matrix. Specifically, at each round t , the algorithm updates the covariance matrix as

$$A_{t+1} = A_t + \mathbf{x} \cdot \mathbf{x}^\top \quad (5)$$

and updates the weight vector

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y_t A_{t+1}^{-1} \mathbf{x}_t \quad (6)$$

The core idea behind this update rule is to precondition the gradient direction based on past observations. The matrix A_{t+1} encodes the directions in which input vectors have been observed. If the current input $\mathbf{x}_t$ lies along a direction frequently encountered in previous rounds, the inverse covariance matrix attenuates the update magnitude. Conversely, when $\mathbf{x}_t$ points in a relatively rare or new direction, the algorithm assigns a larger update.

Online Newton Step (ONS) The Online Newton Step (ONS) algorithm extends Online Gradient Descent (OGD) by incorporating second-order information to adapt learning rates both per-coordinate and over time. It maintains a matrix A_t that approximates the Hessian by accumulating outer products of past gradients

$$A_{t+1} = A_t + g_t g_t^\top, g_t = \nabla f_t(\mathbf{w}_t) \quad (7)$$

The weight vector is updated as

$$\mathbf{w}_{t+1} = \Pi_{\mathcal{D}}(\mathbf{w}_t - \eta A_t^{-1} g_t) \quad (8)$$

where $\Pi_{\mathcal{D}}$ denotes the projection onto the feasible domain $\mathcal{D}$ under the Mahalanobis distance induced by A_t . This update rule leverages curvature information to scale and orient the gradient, enabling more informed and efficient updates, especially in ill-conditioned or anisotropic loss landscapes.

Confidence-Weighted Learning (CW) CW maintains a Gaussian distribution over the weight vector rather than a single deterministic estimate. Specifically, it assumes

$$\mathbf{w}_t \sim \mathcal{N}(\mu_t, \Sigma_t) \quad (9)$$

where μ_t represents the mean and Σ_t the covariance matrix of the weight distribution at round t . Predictions are made based on the confidence-weighted margin

$$\hat{y}_t = \text{sign}(\mathbb{E}_{\mathbf{w} \sim \mathcal{N}(\mu_t, \Sigma_t)}[\mathbf{w}^\top \mathbf{x}_t]) = \text{sign}(\mu_t^\top \mathbf{x}_t) \quad (10)$$

with the associated prediction variance given by

$$\text{Var}(\mathbf{w}^\top \mathbf{x}_t) = \mathbf{x}_t^\top \Sigma_t \mathbf{x}_t \quad (11)$$

At each step, CW selects a new distribution $\mathcal{N}(\mu_{t+1}, \Sigma_{t+1})$ by solving the following constrained optimization problem

$$\min_{\mu, \Sigma} D_{\text{KL}}(\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(\mu_t, \Sigma_t)) \quad (12)$$

with constraint

$$\Pr_{\mathbf{w} \sim \mathcal{N}(\mu_{t+1}, \Sigma_{t+1})} [y_t(\mathbf{w}^\top \mathbf{x}_t) \geq 0] \geq \eta \quad (13)$$

where $D_{\text{KL}}(\cdot, \cdot)$ denotes the KL divergence and $\eta \in (0.5, 1)$ is a user-defined confidence threshold. This formulation ensures that the updated distribution remains close to the

previous one while enforcing a probabilistic margin constraint. The update rules for the mean and covariance are given in closed form

$$\mu_{t+1} = \mu_t + \alpha_t y_t \Sigma_t \mathbf{x}_t \quad (14)$$

$$\Sigma_{t+1}^{-1} = \Sigma_t^{-1} + 2\alpha_t \beta_t \mathbf{x}_t \mathbf{x}_t^\top \quad (15)$$

where α_t and β_t are computed based on the current margin $\mu_t^\top \mathbf{x}_t$ and the variance. CW provides adaptive, confidence-aware updates that naturally scale learning across different feature dimensions. However, CW may be unstable and sensitive to hyperparameter μ and noisy data.

Adaptive Regularization of Weights (AROW) To make CW more robust against label noise, numerically stable, and computationally efficient. AROW was proposed. Unlike CW, which imposes a hard probabilistic constraint, AROW uses a regularized loss minimization approach. It solves the following optimization problem

$$\min_{\mu, \Sigma} D_{\text{KL}}(\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(\mu_t, \Sigma_t)) + C \cdot \ell_t(\mu) \quad (16)$$

where $\ell_t(\mu)$ is a squared hinge loss $\max(0, 1 - y_t \mu^\top \mathbf{x}_t)^2$ and $C > 0$ is a regularization parameter controlling trade-off. This objective allows update only if the prediction is poor, and only as much as needed. Same as CW, in AROM both mean and covariance can be updated in closed-form

$$\mu_{t+1} = \mu_t + \alpha_t y_t \Sigma_t \mathbf{x}_t \quad (17)$$

$$\Sigma_{t+1} = \Sigma_t - \frac{\Sigma_t \mathbf{x} \mathbf{x}_t^\top \Sigma_t}{v_t + \frac{1}{2C}} \quad (18)$$

where

$$\alpha_t = \frac{\ell_t}{v_t + \frac{1}{2C}} \quad (19)$$

If the model is already confident, update is small, otherwise, the update is large. Covariance shrinks in the direction of the current example $\mathbf{x}_t$, reflecting increased certainty after seeing it.

Other Second-Order Algorithms There are also many other variants of second-order algorithms such as soft-confidence algorithm for multi-class classification, active learning, structured prediction, IELLIP, NAROW, and NHERD.

3) *Kernel-Based Online Learning*: Kernelized learning extends linear algorithms to high or infinite-dimensional feature spaces by employing a kernel function

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}') \quad (20)$$

which implicitly computes inner products without explicitly computing the transformation. This enables the construction of nonlinear decision boundaries while keeping computational efficiency. In online learning, kernelization allows classical algorithms such as Perceptron and Passive-Aggressive to perform nonlinear updates using kernel evaluation.

Kernel Perceptron The Kernel Perceptron extends the classic Perceptron to nonlinearly separable problems by representing the decision function as a weighted sum over support

vectors instead of a single weight vector. At round t , the prediction is given by

$$\hat{y}_t = \text{sign} \left(\sum_{(i, y_i) \in \mathcal{M}} y_i K(\mathbf{x}_i, \mathbf{x}_t) \right) \quad (21)$$

where $\mathcal{M}_t$ is the set of past mistake examples, which is updated when new error occurs. If the data is linearly separable in the associated Reproducing Kernel Hilbert Space (RKHS) with margin γ and bounded norm $R = \max_t |\phi(\mathbf{x}_t)|$, the algorithm makes at most $M \leq \frac{R^2}{\gamma^2}$ mistakes. Despite its theoretical appeal, the Kernel Perceptron suffers from increasing computational cost, as the number of support vectors grows linearly with the number of mistakes.

Kernelized Online Gradient Descent (Kernel OGD)

Kernel Online Gradient Descent (Kernel OGD) generalizes the standard OGD algorithm to Reproducing Kernel Hilbert Spaces (RKHS), enabling online convex optimization over nonlinear function classes defined implicitly by a kernel function. Instead of maintaining a linear weight vector, Kernel OGD maintains a function $f_t \in \mathcal{H}$, which can be used for both classification and regression. Leveraging the kernel trick, the function f_t is represented as a weighted sum of kernel evaluations over past instances:

$$f_t(\cdot) = \sum_{i=1}^{t-1} \alpha_i K(\mathbf{x}_i, \cdot) \quad (22)$$

where α_i denotes the weight associated with each example $\mathbf{x}_i$. Given a convex loss function $\ell_t(f) = \ell(f(\mathbf{x}_t), y_t)$, the functional update at round t is

$$f_{t+1}(\cdot) = f_t(\cdot) - \eta_t \nabla \ell(f_t(\mathbf{x}_t), y_t) \cdot K(\mathbf{x}_t, \cdot) \quad (23)$$

which corresponds to appending a new support vector $\mathbf{x}_t$ with coefficient $\alpha_t = \eta_t \nabla \ell(f_t(\mathbf{x}_t), y_t)$.

Budgeted Kernel Online Learning Kernel-based online algorithms suffer from scalability issues, as the number of support vectors (SVs) typically grows linearly with the number of rounds, resulting in prediction and update costs of $\mathcal{O}(T)$ at round T . This renders them impractical for long-term or streaming applications. Budgeted kernel learning mitigates this by enforcing a fixed budget B , ensuring that the support set $\mathcal{S}_t$ satisfies $|\mathcal{S}_t| \leq B$ at all times. The prediction function is then given by

$$f_t(\cdot) = \sum_{i \in \mathcal{S}_t} \alpha_i K(\mathbf{x}_i, \cdot), \quad \text{s.t. } |\mathcal{S}_t| \leq B \quad (24)$$

When an update would exceed the budget, a maintenance strategy is applied to remove or compress existing SVs. Most common strategy is Forgetron that removes the oldest SV (FIFO) and rescales coefficients to control forgetting error. There are other strategies such as Projectron, Randomized Budget Perceptron (RBP), Budgeted Passive-Aggressive (BPA), and Core Set Compression.

Functional Approximation An alternative and efficient approach to improving the scalability of kernel-based online

learning is functional approximation, which approximates the kernel function using a fixed-dimensional feature mapping. Specifically, the kernel is approximated by

$$K(\mathbf{x}, \mathbf{x}') \approx \tilde{\phi}(\mathbf{x})^\top \tilde{\phi}(\mathbf{x}') \quad (25)$$

where $\tilde{\phi}(\cdot) \in \mathbb{R}^D$ is an explicit feature map of dimension D . This approximation transforms the original kernelized model into a linear model in the feature space, enabling efficient prediction and update steps. The prediction function becomes

$$f(\mathbf{x}) = \sum_{i=1}^B \alpha_i K(\mathbf{x}_i, \mathbf{x}) \approx \sum_{i=1}^B \alpha_i \tilde{\phi}(\mathbf{x}_i)^\top \tilde{\phi}(\mathbf{x}) = \mathbf{w}^\top \tilde{\phi}(\mathbf{x}) \quad (26)$$

where $\mathbf{w} \in \mathbb{R}^D$ is a weight vector updated using standard online gradient-based rules. This reformulation reduces the per-round computational cost to $\mathcal{O}(D)$, independent of the number of support vectors. Several methods are used to approximate kernels such as Random Fourier Features (RFF), that construct a feature map

$$\tilde{\phi}(\mathbf{x}) = \sqrt{\frac{2}{D}} [\cos(\mathbf{w}_1^\top \mathbf{x} + b_1), \dots, \cos(\mathbf{w}_D^\top \mathbf{x} + b_D)] \quad (27)$$

where $\mathbf{w}_i \sim p(\omega)$, the Fourier transform of the kernel. There are also other approximation methods such as Nystrom Approximation and Tensor Sketch.

There exist many other online supervised learning algorithms not covered above, yet they remain important. **Regularization-based methods** explicitly control model complexity by incorporating penalty terms into the optimization objective, enhancing model robustness in adversarial settings. **Ensemble methods** such as online bagging, boosting, and random forests maintain a collection of learners that are updated concurrently. By aggregating their predictions, these methods improve overall accuracy and stability, reducing the variance associated with a single model. Additionally, recent advances have introduced adaptive learning rate techniques, hybrid approach feature selection mechanisms, and methods tailored for structured prediction or imbalanced data. These diverse approaches expand the landscape of online learning and continue to evolve to meet the demands of complex, real-world applications.

B. Online Semi-Supervised Learning

Online semi-supervised learning (SSL) operates in streaming environments where only a subset of data points is labeled, while the majority remain unlabeled. It leverages unlabeled data to enhance predictive accuracy under single-pass and bounded memory constraints, particularly valuable in real-time applications where labeling is costly or delayed, such as fraud detection or sensor networks.

1) *Online Self-Training*: Online self-training, also known as pseudo-labeling, is a prominent class of online SSL algorithms. These methods bootstrap supervision from unlabeled data by generating "pseudo-labels" for instances where the model's prediction exhibits high confidence. The core idea

is to augment the limited set of true labels with these high-confidence predictions, thereby providing additional training signals.

The general framework operates on a data stream $\{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$, where the true label y_t is available for a probability p . For each incoming point $\mathbf{x}_t$, the current model f_t generates a prediction $\hat{y}_t = f_t(\mathbf{x}_t)$ and an associated confidence score $c_t = \text{conf}(f_t, \mathbf{x}_t)$. The model is updated if either the true label y_t is provided or the confidence c_t exceeds a predefined threshold θ .

$$\text{Update} = \begin{cases} \text{Yes,} & \text{if } y_t \text{ known or } c_t \geq \theta \\ \text{No (skip or buffer),} & \text{otherwise} \end{cases}$$

Representative techniques include Online Self-Training Perceptron, confidence-thresholded pseudo-labeling, and streaming self-training with budget management. Other advances include mixed-type streaming data [1] and vector quantization [2].

2) *Online Manifold Methods*: This approach is predicated on the manifold hypothesis, which posits that high-dimensional data $\mathbf{x}_t \in \mathbb{R}^d$ lies on or near a lower-dimensional manifold $\mathcal{M} \in \mathbb{R}^n$. It further assumes that the labeling function varies smoothly along this manifold. Online manifold methods incrementally construct a graph-based approximation of the manifold structure from the data stream and use it to regularize the learning process.

The learning objective at each step typically minimizes a composite loss function over a set of labeled instances $\mathcal{L}_t$ and a buffer of unlabeled instances $\mathcal{U}_t$:

$$\min_f \left(\sum_{(x_i, y_i) \in \mathcal{L}_t} \ell(f(\mathbf{x}_i), y_i) + \lambda \|f\|_{\mathcal{H}}^2 + \gamma \mathcal{R}_{\text{manifold}}(f, \mathcal{G}_t) \right) \quad (28)$$

Here, $\ell(\cdot, \cdot)$ is a supervised loss function, $\|f\|_{\mathcal{H}}^2$ is a complexity regularizer on the function f , and $\mathcal{R}_{\text{manifold}}$ is a manifold regularizer defined over a similarity graph $\mathcal{G}_t$ constructed from $\mathcal{L}_t \cup \mathcal{U}_t$. A common form for this regularizer is the graph Laplacian regularizer:

$$\mathcal{R}_{\text{manifold}}(f, \mathcal{G}_t) = \sum_{i,j} W_{ij}^{(t)} (f(\mathbf{x}_i) - f(\mathbf{x}_j))^2 = f^\top L_t f \quad (29)$$

where $L_t = D_t - W_t$ is the graph Laplacian of $\mathcal{G}_t$. Notable examples include Online Laplacian Regularized Least Squares (LapRLS) and Online Label Propagation.

3) *Online Regularization-Based methods*: This family of methods incorporates unlabeled data into the learning process by introducing explicit regularization terms to the online objective function. These terms enforce desired properties on the model's behavior, such as consistency under perturbations or low-entropy (high-confidence) predictions on unlabeled data. The general objective function at time t can be expressed as:

$$\mathcal{L}_t(f) = \ell(f(\mathbf{x}_t), y_t) + \lambda R_{\text{sup}}(f) + \gamma R_{\text{unsup}}(f, \mathbf{x}_t) \quad (30)$$

where ℓ is the loss function, R_{sup} is standard regularizer such as $\|f\|^2$ or weight decay. R_{unsup} is the unsupervised regularizer applied to unlabeled data, λ, γ are the regularization coefficients. At each time, the model receives $\mathbf{x}_t$, optionally y_t , and performs an online update based on the objective above. Key techniques in this category include Entropy Minimization(EM), Consistency Regularization(CR), Multi-View Co-Regularization(MCR), and Hybrid Regularization(HR), each with different regularizer as follows:

TABLE I
REGULARIZERS

Technique	Regularizer
EM	$R_{\text{unsup}}(f, \mathbf{x}_t) = \mathbb{H}(f(\mathbf{x}_t)) = -\sum_k f_k(\mathbf{x}_t) \log f_k(\mathbf{x}_t)$
CR	$R_{\text{unsup}}(f, \mathbf{x}_t) = \ f(\mathbf{x}_t) - f(\mathbf{x}_t + \delta_t)\ ^2$
MCR	$R_{\text{unsup}} = \ f^{(1)}(\mathbf{x}_t) - f^{(2)}(\mathbf{x}_t)\ ^2$
HR	$R_{\text{unsup}} = \alpha \mathbb{H}(f(\mathbf{x}_t)) + \beta \ f(\mathbf{x}_t) - f(\mathbf{x}_t + \delta_t)\ ^2$

C. Online Unsupervised Learning

Online unsupervised learning is a class of algorithms that learn patterns or structures from unlabeled data arriving sequentially over time. Unlike supervised learning, these methods operate without explicit feedback, aiming to uncover underlying distributions, clusters, or latent representations in a streaming setting. Common tasks include online clustering, density estimation, dimensionality reduction, and anomaly detection.

1) *Online Clustering*: Online clustering refers to a class of algorithms that incrementally group streaming data points into clusters without requiring access to the entire dataset. The objective is to discover and maintain coherent cluster structures in real time, under constraints on memory and per-step computation. Given a stream of unlabeled data points $\mathbf{x}_1, \mathbf{x}_2, \dots$, the algorithm outputs either cluster assignments or representative prototypes such as centroids or medoids.

The general framework is as follows. Upon the arrival of a new data point $\mathbf{x}_t$, its distance to existing cluster representatives (e.g. centroids) is computed. A decision rule, often based on a distance threshold θ determines:

$$\text{Action}(\mathbf{x}_t) = \begin{cases} \text{Assign } \mathbf{x}_t \text{ to } C_j & \text{if } d(\mathbf{x}_t, \mu_j) \leq \theta \\ \text{Create new cluster with } \mathbf{x}_t & \text{otherwise} \end{cases}$$

If assigned, the centroid of C_j is updated to incorporate $\mathbf{x}_i$.

Prototype-Based Methods maintain a fixed number of representative prototypes. A classic example is *online k-means*, which maintains k centroids $\mu_1, \dots, \mu_k$. For each new data point $\mathbf{x}_t$, the nearest centroid j is updated as:

$$\mu_j \leftarrow \mu_j + \eta_t(\mathbf{x}_t - \mu_j) \quad (31)$$

where η_t is a learning rate. Variants include streaming k-means and mini-batch k-means.

Probabilistic Model-Based Methods represent clusters via statistical models, allowing soft assignments and probabilistic

interpretation. The most common approach is the *Online Gaussian Mixture Model (GMM)*, which maintains means, covariances, and mixture weights of components, updating them via incremental Expectation-Maximization (EM) or stochastic variational inference.

Coreset and Sketch-Based Methods operate in two phases.

i) **Online Micro-Clustering** processes the data stream to create a compressed representation, known as a coreset or sketch such as count, linear sum, and squared sum. ii) **Offline Macro-Clustering** applied conventional offline clustering algorithms (e.g. k-means) to this coreset to derive the final macro-clusters.

Other approaches include **Density-Based Methods**, which form clusters based on local density and neighborhood connectivity, and **Nonparametric or Dynamic Clustering**, which adaptively estimate the number of clusters and track cluster evolution over time

2) **Online Dimensionality Reduction**: Online dimensionality reduction was developed for data with large feature dimensions. It learns to transform high-dimensional streaming data into a lower-dimensional representation incrementally, without needing the entire dataset at once. Its goal is to find a compact data representation on-the-fly while preserving as much important information as possible.

Online density estimation involves incrementally learning a probability distribution $p(\mathbf{x})$ over data that arrives sequentially. Unlike batch methods that require access to the full dataset, online methods update the density model in real time, typically under constraints on memory and computation.

A general framework initializes a low-dimensional basis B —either randomly or from an initial data batch. Upon receiving a new data point $\mathbf{x}_t \in \mathbb{R}^d$, the algorithm computes its low-dimensional representation:

$$\mathbf{z}_t = \text{project}(\mathbf{x}_t, B^{t-1}) \quad (32)$$

and may reconstruct the original data as:

$$\hat{\mathbf{x}}_t = \text{reconstruct}(\mathbf{z}_t, B^{t-1}) \quad (33)$$

The basis B^{t-1} is then updated to B^t based on $\mathbf{x}_t$ to minimize reconstruction error. The update step is designed to be efficient and memory-bounded.

Online dimensionality reduction techniques can be categorized into three main types:

(i) **Linear Subspace Tracking**: These methods assume the data lies near a low-dimensional linear subspace and incrementally update the basis to reflect changes. Algorithms such as *Online Principal Component Analysis (PCA)* and *Incremental Singular Value Decomposition (SVD)* track the top- k eigenvectors of the data covariance matrix using gradient-based or matrix-factorization updates. Other techniques include online *Independent Component Analysis (ICA)*.

(ii) **Probabilistic Models**: These approaches model data generation from a latent low-dimensional space with added noise. A common formulation is probabilistic PCA $\mathbf{x}_t = \mathbf{W}\mathbf{z}_t + \epsilon_t$ where $\mathbf{W}$ is updated incrementally using online Expectation-Maximization (EM) or variational inference.

Extensions include *Online Factor Analysis*, which allows a diagonal noise covariance, and *Online Variational Inference* for more flexible Bayesian latent variable models.

(iii) **Nonlinear Embedding**: These methods preserve non-linear structures such as local neighborhoods or global similarity in the low-dimensional space. Techniques include *Incremental Kernel PCA*, which applies the kernel trick for non-linear projection, as well as *Online Nystrom Approximation* and *Online Manifold Learning*. Neural approaches such as *Online Autoencoders* and contrastive methods like *SimCLR* or *MoCo* variants extend representation learning to dynamic, non-linear environments.

3) **Online Density Estimation**: Online density estimation involves incrementally learning a probability distribution $p(\mathbf{x})$ over data that arrives sequentially. Unlike batch methods that require access to the full dataset, online methods update the density model in real time, typically under constraints on memory and computation. It can be broadly categorized into the following classes:

(i) **Parametric Online Estimation** assumes a known parametric form (e.g., Gaussian or exponential family) and update the distribution parameters recursively. For example, online maximum likelihood estimation for a Gaussian distribution updates the mean and covariance as:

$$\mu_{t+1} = \mu_t + \eta_t(\mathbf{x}_t - \mu_t) \quad (34)$$

$$\Sigma_{t+1} = \Sigma_t + \eta_t[(\mathbf{x}_t - \mu_t)(\mathbf{x}_t - \mu_t)^\top - \Sigma_t] \quad (35)$$

where η_t is a learning rate. Other methods include recursive Bayesian estimation, which updates posterior distributions over parameters using Bayes' rule with each new observation.

(ii) **Nonparametric Online Estimation** makes no strong distributional assumptions and instead approximate $p(\mathbf{x})$ directly from data. A typical method is online kernel density estimation (KDE):

$$p_t(\mathbf{x}) = \frac{1}{t} \sum_{i=1}^t K_h(\mathbf{x} - \mathbf{x}_i) \quad (36)$$

where K_h is a kernel with bandwidth h . In online settings, KDE employs techniques such as kernel merging, budget maintenance, and exponential forgetting to ensure scalability. Other examples include histogram-based methods that maintain frequency counts in discretized input space.

(iii) **Bayesian Nonparametric Methods** place infinite-capacity priors (e.g., Dirichlet Process Mixtures) over the data distribution, allowing flexible adaptation as new data arrives. Updates are typically performed using online variational inference or sequential Monte Carlo methods. Notable examples include Online Dirichlet Process Gaussian Mixture Models (DP-GMM) and Online Hierarchical Pitman-Yor Processes.

4) **Anomaly Detection**: Online anomaly detection (AD) involves identifying rare or abnormal patterns in streaming data without labeled supervision. The objective is to detect deviations from expected behavior in real time, under constraints of limited memory and low latency, while remaining

robust to evolving data distributions (i.e., concept drift). AD algorithms can be broadly categorized as follows:

Distance-Based Methods define normality based on geometric proximity to known data instances, where anomalies lie far from typical clusters or neighborhoods. Typical approaches include online k -nearest neighbors (KNN), incremental clustering (e.g., micro-clusters with outlier scores), and distance-to-centroid models (e.g., online k -means with thresholding). These methods are intuitive but can be sensitive to high-dimensional noise and require efficient nearest neighbor computation.

Density-Based Methods assume that anomalies occur in low-density regions and estimate density online using histograms, kernel density estimation (KDE), or sketching techniques. Examples include online KDE with forgetting factors, histogram-based counting, and stochastic density-ratio estimation. However, scalability to high-dimensional data or long histories remains a challenge.

Model-Based Methods learn predictive models online to capture normal behavior, flagging instances with high reconstruction error or low likelihood as anomalies. Common approaches include online autoencoders, which reconstruct inputs to identify high-error anomalies, and online one-class SVM (OC-SVM), which incrementally estimates the boundary of the normal region.

Tree-Based Methods use recursive partitioning to isolate anomalies with fewer splits than normal instances. Online Isolation Forest (iForest), for example, incrementally builds random trees and assigns anomaly scores based on average path lengths. Half-Space Trees extend this idea by adapting to concept drift using half-space partitions.

Anomaly detection (AD) remains a popular area of research, yet several challenges persist. One major issue is concept drift, where the definition of "normal" behavior evolves over time, necessitating adaptive mechanisms. Although online sliding window techniques have been proposed to address this, robust handling of drift remains difficult. Another challenge is label scarcity, as most data streams are fully unsupervised with little to no feedback. Additionally, high false positive rates pose a risk of alarm fatigue, making sensitivity tuning critical. Finally, real-time constraints require models to operate with low latency and bounded memory, further complicating deployment in streaming environments.

D. Online Deep Learning

Online deep learning (ODL) extends deep neural architectures—e.g., convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Transformers—to streaming settings. In contrast to earlier online methods that employ shallow models (e.g., Winnow), ODL extracts hierarchical representations from high-dimensional data while updating model parameters in real time. This capability is crucial for applications that demand both expressive modeling and continual adaptation, such as banking fraud detection, speech recognition, video analytics, and autonomous driving.

Current research in ODL centers on two complementary themes: (i) *Continual-learning strategies*, which mitigate catastrophic forgetting through mechanisms such as rehearsal buffers, parameter regularization, and dynamic expansion; and (ii) *Neural-network architectures*, which instantiate these strategies using architectures such as CNNs, RNNs, graph neural networks, and sequence Transformers designed for low-latency, memory-bounded updates.

1) *Continual Learning Strategy*: **Replay-based methods** address the challenge of catastrophic forgetting in continual learning by storing a subset of past examples and replaying them alongside new data during model updates. In the online deep learning setting, these methods are designed to operate under strict memory constraints, using a limited buffer to maintain a small but representative subset of the data stream. They are especially effective when task boundaries are not explicitly known or when the data distribution changes over time (concept drift).

The general framework is as follows. At each time t , the learner receives a new example $(\mathbf{x}_t, y_t)$ and updates the model using both $(\mathbf{x}_t, y_t)$ and a mini-batch $\mathcal{B}_t$ sampled from replay buffer $\mathcal{M}_t$. The loss function typically combines both current and past data

$$\mathcal{L}_t(\theta) = \ell(f_\theta(\mathbf{x}_t), y_t) + \frac{1}{|\mathcal{B}_t|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{B}_t} \ell(f_\theta(\mathbf{x}_i), y_i) \quad (37)$$

After the update, the buffer $\mathcal{M}_t$ is updated using a buffer management policy, such as reservoir sampling or class-balanced replacement.

Several typical examples include Incremental Classifier and Representation Learning (iCaRL) [3], Continual learning with deep generative replay [4], Gradient episodic memory for continual learning, [5], Memory efficient experience replay for streaming learning [6], and Gradient based sample selection for online continual learning [7], and Online continual learning with maximal interfered retrieval [8]. Recently, more advanced work developed more complex mechanisms including Rethinking Data Use in Large Language Models [9], A masking, linkage, and guidance framework for online class incremental learning [10], Online Continual Learning via Spiking Neural Networks with Sleep Enhanced Latent Replay [11], Learning on streaming graphs with experience replay [12].

Besides, replay methods are also extensively used in deep reinforcement learning such as Recurrent experience replay in distributed reinforcement learning [13], Prioritized experience replay [14], and Hindsight experience replay [15].

Parameter-Isolation Methods address catastrophic forgetting in continual learning by dedicating specific subsets of model parameters to different tasks or data distributions. Instead of replaying past data or constraining gradients, these methods modularize or dynamically adapt the model architecture to preserve previously learned knowledge. In the online deep learning setting, parameter-isolation methods are especially attractive due to their task-incremental compatibility and low interference between tasks.

The core idea is as follows. At time step t , the learner receives a new data instance $(\mathbf{x}_t, y_t)$ from a potentially new task or domain. Rather than updating all weights of the model f_θ , parameter-isolation ensure that only a subset of parameters are activated and updated, often task-specific. The overall model may be viewed as

$$f_\theta(\mathbf{x}) = f_{\theta^{(1)}}(\mathbf{x}) + \dots + f_{\theta^{(k)}}(\mathbf{x}) \quad (38)$$

where $\theta^{(i)}$ denotes parameters dedicated to the i^{th} task.

Representative work includes Progressive neural networks [16], Pathnet [17], Piggyback Networks [18], PackNet [19], and Supermasks in Superposition [20]. More advanced approaches include Continual Learning with Hypernetworks [21], Additive Parameter Decomposition [22], Neural Pruning [23], and Dynamic Graph [24].

Dynamic-Architecture Methods Dynamic-architecture methods address the challenge of catastrophic forgetting by adapting the network structure over time in response to new data. Instead of fixing the model architecture upfront, these methods dynamically expand, prune, split, or route neural modules to accommodate new tasks or changing distributions. In the online deep learning setting, they enable continual adaptation while maintaining model flexibility and efficiency.

The problem setup becomes as follows. The model at time t is parameterized by θ_t and structured as f_{θ_t} . As new data $(\mathbf{x}_t, y_t)$ arrives, the model dynamically determines whether to reuse existing parameters or expand the architecture by adding new units or modules. The optimization thus becomes a structural search

$$\min_{\theta_t, \mathcal{A}_t} \mathcal{L}(f_{\theta_t}^{\mathcal{A}_t}(\mathbf{x}_t), y_t) + \Omega(\mathcal{A}_t) \quad (39)$$

where $\mathcal{A}_t$ is the architecture at time t , and Ω is a regularizer on complexity.

Typical examples include Lifelong learning with dynamically expandable networks [25], Adaptive Depth with Ensembled Learning and Hedge propagation [26], Adaptive Structural Learning of Artificial Neural Networks (Adanet) [27], Dynamically expandable representation for class incremental learning (Der) [28], Hierarchically Gated Experts for Efficient Online Continual Learning [29], and Enhancing CNN incremental learning capability with an expanded network [30].

There are still many other approaches that were not covered above, yet remain significant such as Meta Learning for Continual Learning [31] [32] [33], and regularization methods [34] [35] [36] [37] [38].

2) *Neural Networks Architecture*: Following is a discussion of different architectures that online deep learning is instantiated, which includes Convolutional Neural Networks, Recurrent Neural Networks, Transformer, and Graph-Neural Networks.

Convolutional Neural Networks (CNNs) are a specialized type of deep neural network designed to process grid-like data, most commonly used for image and video analysis, but also applicable to audio and time-series data. Catastrophic forgetting is a key issue for CNNs and many online approaches

discussed above can be instantiated using this architecture such as Memory Replay [4] [39] [40], developmental memory [41], Deep Streaming Linear Discriminant Analysis (Deep SLDA) [42]. More advance techniques include time-channel maps [43], and Tree-hierarchical CNNs [44]. As expected, most of the work is for image classification or video generation.

Recurrent Neural Networks (RNNs) is a class of neural architectures designed for sequential data, such as time series, speech, and text, which is naturally suitable for an online learning setting. Some representative works include LSTM-based Online learning that develops an adaptive "forget gate" to reset the cell at a certain time [45] or uses Online Gradient Descent and Extended Kalman Filter [46]. Some approaches use an energy-efficient operator with the exponentiated gradient method to make LSTM energy efficient for online learning [47].

Transformer Transformer networks are deep learning architectures based entirely on self-attention mechanisms, originally introduced for sequence modeling tasks [48]. It is often compared with RNNs, which model sequences step-by-step through recurrence, Transformer processes inputs in parallel via attention, allowing more efficient scaling and superior long-range dependency modeling. In Online Transformer, some representative techniques applied in streaming speech recognition include Time-Restricted Self-Attention on Encoder and Triggered Attention on Decoder [49], or integrated transducer framework in Transformer [50]. Some methods introduce a token expansion mechanism to enhance its ability of continual learning [51], or use inter-task attention to slow down the drift of important attention [52].

Graph-Neural Networks (GNNs) is a special neural network designed for graph data structures which are powerful for representing relationships between entities [53] [54] in many real-world systems such as social networks, communication logs, financial transactions, and recommendation engines. In many applications, the graph structure evolves over time, with new nodes and links arriving continuously. Recent work [55] demonstrates that pre-trained GNNs can efficiently handle incremental updates and often outperform models retrained from scratch, underscoring their potential in online learning settings. To address the challenges posed by streaming graph data, a class of online deep learning models—most notably Temporal Graph Neural Networks (TGNNs)—has been developed [56].

Initial approaches to learning on dynamic graphs often involved applying static models to aggregated graph structures, thereby ignoring their temporal evolution [57] [58]. Some methods attempted to address this limitation by encoding each graph snapshot through aggregation [59] [60] and modeling the temporal evolution using time-series models, such as recurrent neural networks (RNNs) [61] [62] [63], or by enforcing temporal smoothness constraints [64] [65] [66] [67]. Other efforts explored random walk-based techniques [68] [69] [70] [71] and spatio-temporal graph representations [72] [73]. A more structured line of research introduced the concept of discrete-time dynamic graphs (DTDGs), where the dynamic graph is modeled as a sequence of graph snapshots [74] [75]

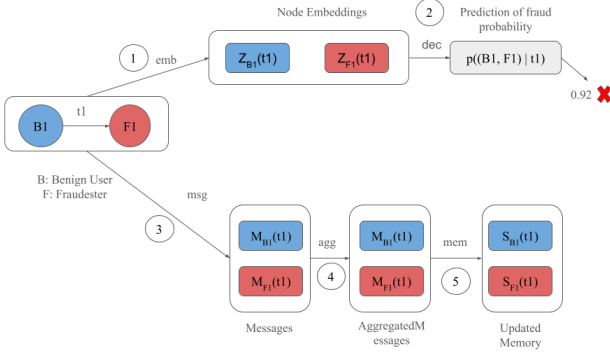


Fig. 2. Illustration of a temporal graph neural network for fraud detection

[76] [77]. However, this discrete representation fails to capture the continuous nature of interactions in many real-world systems, such as social networks or financial transactions.

Consequently, research has shifted towards Continuous-Time Dynamic Graphs (CTDGs), which model interactions as a timestamped sequence of events. Early contributions include dynamic knowledge graphs [78] [79] [80] [81] and temporal extensions of random walk models that constrain transition probabilities over time [82] [83] [84]. Foundational work in dynamic embeddings established methods for updating node representations over time. Building on this, a predominant architectural pattern emerged that utilizes a sequence-based method with node-wise memory module, typically implemented with a Recurrent Neural Network (RNN), to update representations upon new interactions [85] [86]. This approach led to several influential models, including DyRep [87], Jodie [88], and TGAT [89].

More recently, the Temporal Graph Network (TGN) framework was proposed as a unified architecture for learning on continuous-time dynamic graphs (CTDGs) [56]. TGN made a significant contribution by showing that many prior memory-based models are special cases of its general framework, thus offering a unifying perspective for a key class of temporal GNNs. For instance, Jodie can be viewed as a TGN variant that incorporates a time projection embedding module; TGAT corresponds to a TGN without the memory module and employs graph attention within the embedding function; and DyRep aligns with TGN by computing messages via graph attention over destination node neighborhoods.

IV. APPLICATION OF ONLINE LEARNING

The ability of online learning algorithms to process data incrementally and adapt continuously makes them suitable for modern applications characterized by high-velocity, non-stationary data streams and the need for real-time decision-making. Unlike batch learning, which requires periodic re-training on static datasets, online learning could be specifically effective in dynamic environments where information arrives sequentially and concepts may drift over time. This inherent adaptability shows significant value across a diverse spectrum

of domains. Key application area mainly leveraging online learning include:

- 1) **Fraud Detection:** Requiring immediate response to evolving fraudulent patterns in financial transactions and user activities.
- 2) **Recommendation System & Personalization:** Continuously adapting to shifting user preference and item popularity to deliver relevant suggestions in real-time.
- 3) **Cybersecurity and Intrusion Detection:** Identifying novel threats and anomalous network behavior as they emerge in fast-paced network traffic.
- 4) **Stock Market Prediction & Portfolio Optimization:** Reacting instantaneously to market fluctuations and news events to inform trading strategies.
- 5) **Smart Cities, IOT, & Sensor System:** Analyzing continuous data flows from various sensors for traffic management, environmental monitoring, predictive maintenance, and resource optimization.
- 6) **Healthcare:** Enabling real-time patient monitoring, adaptive treatment planning, and early detection of health diseases based on streaming data.

The subsequent sections will delve deeper into these critical applications, starting with a focused analysis of **Fraud Detection**, highlighting the specific challenges and online learning approaches employed in this domain.

A. Online Fraud Detection

Fraud, in its general sense, is an intentional act of deception for personal or financial gain. It involves the manipulation of information, identity, or systems to mislead individuals and organizations [90]. Among its various manifestations, financial fraud represents a particularly prevalent and high-impact domain. This category includes deliberate acts of deception within financial transactions, including credit card fraud, financial statement fraud, insurance claims fraud, and many other [91]. Fraudsters exploit vulnerabilities in financial systems to misappropriate funds, manipulate records, or access confidential information, leading to substantial economic losses and reputational damage. Detecting such fraud is challenging due to its evolving nature and the scarcity of labeled data [92].

Online learning presents a robust framework for addressing these issues. By processing data sequentially and updating models in real time, online algorithms can continuously adapt to novel fraud patterns [93]. This capacity for rapid adaptation, combined with their computational efficiency under strict memory and latency constraints, makes them particularly well-suited for deployment in high-throughput financial systems such as payment gateways and core banking platforms [94].

This review focuses on the application of online learning to financial fraud detection. The subsequent sections are organized as follows: first, we define key fraud typologies; second, we survey relevant online learning techniques for their detection; and third, we discuss the datasets and evaluation metrics standard to this domain.

TABLE II
COMPARISON OF ONLINE LEARNING FAMILIES ACROSS EIGHT PRACTICAL DIMENSIONS

Method	Core Idea	Strengths	Weaknesses	Concept Drift Adaptation	Best-fit Use Cases
First-Order	Incremental updates with first-order gradients	Simple, fast, scalable, supports windows and decay	Learning rate sensitive, slow to abrupt drift, limited expressiveness	Good with decay and adaptive LR	Recommendation system, text classification, fraud scoring
Second-Order	Maintain curvature information for adaptive steps	Fast convergence, robust with correlation and small data	Heavy compute and memory, approximations needed	Good with decay and uncertainty-aware updates	Low-latency online prediction, sensor calibration
Kernel-Based	Nonlinear decision boundaries with kernels and budgeted support sets	High expressiveness, fits complex boundaries	Support vector growth, pruning needed, kernel sensitive, brittle to drift	Fair with pruning and decay	Nonlinear classification, anomaly rejection under resource limits
Semisupervised	Leverage many unlabeled and few labeled samples with consistency or self-training	Cuts label cost, uses delayed labels	Confirmation bias, pseudo-label drift, higher cost	Fair with forgetting and re-labeling	Content moderation, recommendation system cold-start
Clustering	Incrementally maintain clusters with micro-clusters, density or grids	No labels, natural grouping, supports evolution	Parameter sensitive, high-dimensional issues, split/merge complex	Good with decay and weighting	IoT monitoring, traffic profiling, user segmentation
Density Reduction	Use Coresets, sketching, projections to compress data streams	Memory/compute savings, supports downstream learners	Approximation error, rare pattern loss	Fair with decay	Preprocessing, visualization, load reduction before online training
Density Estimation	Estimate probability density incrementally with Kernel Density Estimation or mixture models	Supports novelty and drift monitoring	Hard in high-dimension, bandwidth sensitive, complex	Good with adaptive bandwidth and decay	Quality monitoring, distribution shift detection
Anomaly Detection	Detect outliers online via rarity, density, distance or trees	No labels, detects unknown fraud and attacks	High false alarms, threshold sensitive	Fair with adaptive thresholds	Fraud/intrusion detection, log monitoring, IoT alerts
Continual	Prevent forgetting via regularization, replay, parameter isolation and distillation	Expressive, multimodal, robust with replay	Heavy compute and memory, instability, forgetting risk	Good with replay and adaptive strategies	Recommendation system, fraud sequences detection, robotics

1) *Fraud Typologies*: Fraud happens in diverse forms across various sectors, each presenting unique detection challenges and opportunities for the application of online machine learning techniques. This section outlines key fraud typologies relevant to this review, categorized by their domain and characteristics.

Credit Card Fraud

Credit card fraud involves the unauthorized use of credit or debit card credentials to conduct illicit transactions [95]. It is broadly classified into two types:

- **Offline Fraud**: The unauthorized use of a physical card that has been lost, stolen, or counterfeited.
- **Online Fraud**: The use of compromised card information (e.g., card number, expiration date, CVV) to perform transactions over the internet, by phone, or via mail order.

The high volume and velocity of electronic payments make this a primary target for fraudsters and a critical application area for real-time, adaptive detection systems.

Financial Statement Fraud

This category refers to the deliberate misrepresentation or omission of information in an organization's financial reports to deceive stakeholders [96]. The primary motivations include inflating share prices, securing favorable financing, or minimizing tax liabilities [97]. Common methods involve:

- Fictitious Revenues
- Improper asset valuation
- Concealment of liabilities and expenses
- Inappropriate disclosures

Unlike high-frequency transactional fraud, financial statement fraud is often a low-frequency, high-impact event orchestrated by internal actors, requiring sophisticated analytical methods to detect anomalies in financial reporting data over time.

Insurance Claim Fraud

Insurance fraud comprises deceptive acts intended to secure illegitimate benefits from an insurance provider [98]. These activities are often categorized as either **opportunistic** (e.g., exaggerating a legitimate claim) or **premeditated** (e.g., staging an automobile collision or billing for non-rendered med-

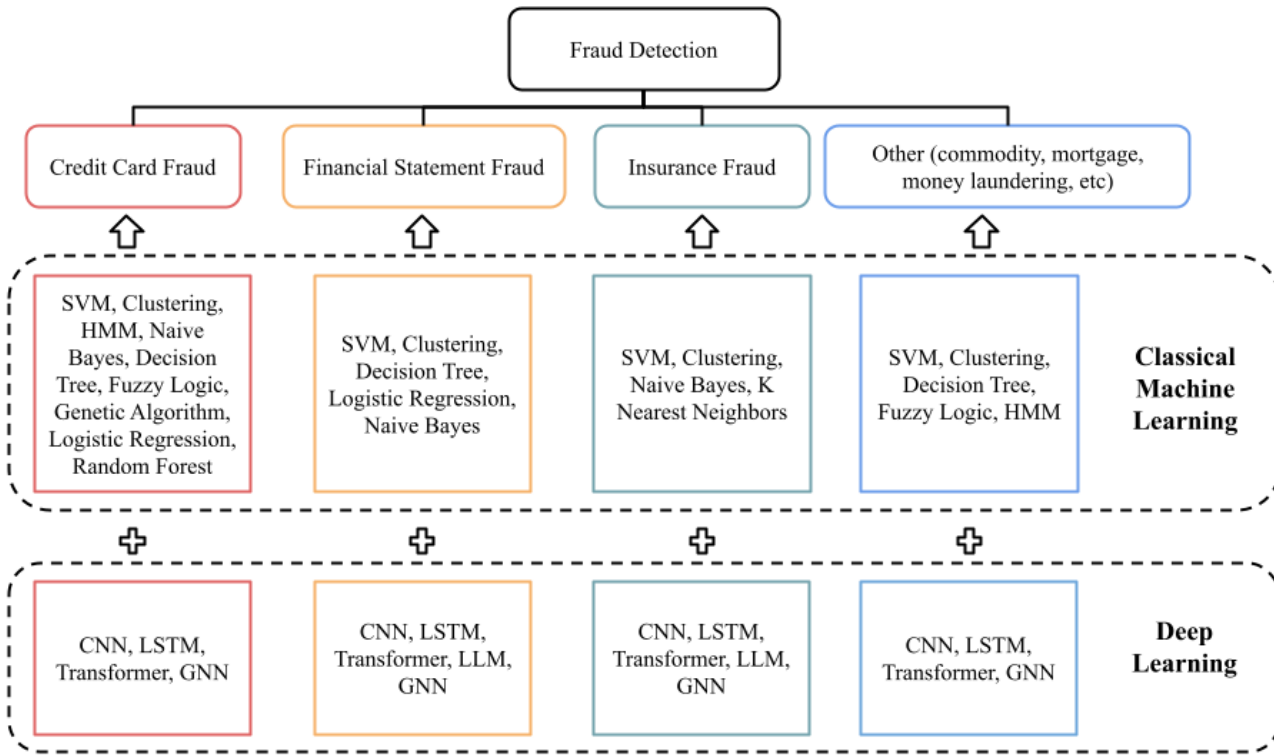


Fig. 3. Fraud detection methods across domains using classical machine learning and deep learning approaches.

ical services). Fraud can be perpetrated by policyholders, claimants, or service providers at various stages, including application, premium payment, or claim submission [99]. Insurance fraud is particularly prevalent in auto and healthcare insurance domains, imposing substantial costs on insurers and raising premiums for legitimate policyholders. The detection challenge arises from the diverse data types involved (textual, numerical, behavioral) and the dynamic nature of fraud schemes, necessitating adaptive, online learning approaches for effective real-time analysis.

Other Fraud Beyond the primary types discussed, several other forms of fraud, both financial and non-financial, are pertinent to the development of advanced detection systems:

- **Commodity & Security Fraud:** Deceptive practices in financial markets, such as insider trading, market manipulation, or false information to influence the price of stocks or commodities [100].
- **Money Laundering:** A process of concealing the origins of illegally obtained money by passing it through a complex sequence of legitimate financial transactions [101].
- **Mortgage Fraud:** Intentional misrepresentation or omission of information on mortgage loan applications to secure financing that would otherwise not be granted [102].
- **Cryptocurrency Fraud:** Deceptive schemes involving digital currencies, including phishing, fraudulent Initial Coin Offerings (ICOs), and exchange hacks [103].

- **Human Trafficking:** Exploitation of individuals through force, fraud, or coercion for purposes such as forced labor or commercial sex, often involving illicit financial transactions [104].
- **E-commerce Click Fraud:** The artificial inflation of clicks on pay-per-click online advertisements, typically executed by automated bots to deplete an advertiser's budget or generate fraudulent revenue for a publisher [105].

Overall, for online fraud detection, the system shall be capable of:

- Sequential Processing Without Retraining
- No Need for Historical Data Storage
- Adaptability to Concept Drift
- Immediate Decision Capability

2) *Online Learning Techniques for Detection:* This section explores different online learning techniques used in fraud detection, ranging from classical approaches such as Support Vector Machines (SVM), Naive Bayes, K-Nearest Neighbors, and Logistic Regression to Deep Learning approaches such as CNNs and TGNs.

Support Vector Machines (SVMs) SVMs are among the most widely used approaches for fraud detection. Most prior research has adopted a batch learning paradigm. For example, [106] applied SVMs to financial statement fraud detection by integrating quantitative financial ratios with qualitative linguistic and textual features. Building on this, [107] incorporated audio features, while [108] employed a One-Class SVM. A

concise overview of SVM-based financial statement fraud detection is provided in [109].

In contrast, adapting SVMs to online learning poses significant challenges, as data typically arrives sequentially and the full dataset is unavailable at initialization. [110] introduced techniques for online SVMs using non-convex loss functions, budget constraints, and online stochastic gradient-based optimization. [111] extended SVMs to an online setting via an Alternating Direction Method of Multipliers (ADMM) optimization framework. Similarly, [112] implemented SVMs on Apache Spark for credit card fraud modeling, with potential applicability to streaming data.

Severe class imbalance is a major challenge in online fraud detection, as fraudulent transactions form only a small fraction of the data. Research has largely centered on credit card fraud due to high transaction volume, continuous data streams, and the need for sub-second decisions. Approaches include Danger Theory-inspired parameter optimization [113], user-specific behavioral profiling [114], and autoencoder-based feature learning with One-Class SVM [115]. Beyond credit cards, adaptive synthetic sampling has been applied to insurance fraud [116], while SVMs on financial and governance indicators have been used for financial statement fraud [117]. Concept drift is another critical issue, addressed via hybrid feature selection and parameter optimization [118] and continuous behavioral profile updates [114].

However, our observation shows that even though SVMs are strong batch classifiers, they're an awkward fit for streaming fraud detection. The root cause lies in global max-margin solution that depends on the entire data distribution. In Kernel form, the hypothesis is (22). The model is a sum over past examples, so adapting to new data often requires revisiting old points. Without storing them, accuracy and guarantees degrade; with them, support vectors grow over time, increasing memory and prediction latency—undesirable for online systems. Budget maintenance can control cost but sacrifices optimality and is fragile under concept drift.

Many fraud patterns are relational (device-account-merchant graphs) or sequence-dependent (customer journeys). A vanilla tabular SVM under-utilizes these dependencies.

Online Clustering

Clustering is another popular technique, in offline setting, it's applied in various domains [119]–[122]. For online setting, compared to Online SVMs, online clustering has been proposed earlier and studied more extensively. [123] provides a general overview of online clustering, while [124] introduces a multi-expert framework that adaptively reweights experts using exponential weighting to minimize regret and track the best clustering strategy over time. Other methods employ nearest-centroid assignment with learning-rate-based updates [125] or density-based incremental updates [126]. Applications include telecommunications fraud detection [127].

Notably, most online clustering research focuses on anomaly detection, an umbrella category encompassing fraud detection as a subset, since anomalies may be benign, whereas fraud is

inherently malicious [128]. This distinction makes fraud detection more challenging. While online clustering is effective at identifying unusual patterns, fraud detection requires evidence of malicious intent. Many anomalies (e.g., new merchant activity, travel, salary bonuses) are harmless, leading to a high false-positive rate when using purely distance-based anomaly scores. Because intent is subtle and fraud prevalence is extremely low, some degree of supervision is typically required to differentiate costly anomalies from benign deviations.

Additional challenges include camouflage—where fraudulent behavior mimics normal patterns and resides within dense clusters—and behavioral drift. Both benign and malicious behaviors can evolve over time (e.g., seasonal changes, customer lifecycle), necessitating continuous centroid updates [92]. Distinguishing benign drift from attack-induced drift often requires labels or domain-specific constraints.

Despite these limitations, online clustering can be valuable for front-end triage, detecting collusive patterns, and cold-start detection. Promising research directions include hybrid approaches that integrate representation learning for feature updates, streaming clustering for anomaly detection, and supervised post-filtering with delayed labels to identify truly malicious activity. Techniques such as soft labeling, active learning, and drift-aware clustering may further enhance performance in fraud detection contexts.

Naive Bayes

The Naive Bayes (NB) classifier is a widely used probabilistic method that naturally adapts to online learning due to its reliance on Bayes' theorem and the maintenance of simple sufficient statistics. Its parameters can be updated incrementally in constant time without retaining historical data, making it one of the most computationally efficient classifiers to “streamify.” Early applications of Bayesian networks in fraud detection include credit card fraud [129] and insurance fraud [130]. Subsequent studies have integrated NB with data-mining techniques for financial statement fraud detection [131], [132], while [109] combined Bayesian networks with ensemble learning to enhance detection using both financial metrics and textual analysis of managerial commentary. In credit card fraud detection, [133] improved Bayesian networks by incorporating Dempster-Shafer theory, and [134] developed a customized Bayesian Network Classifier using a Hyper-Heuristic Evolutionary Algorithm. NB has also shown strong performance in highly skewed datasets [135], and has been applied in domains such as medical insurance fraud [136], cybersecurity intrusion detection [137], and ATM phone scam detection [138].

Despite its suitability for streaming contexts, NB's conditional independence assumption is often violated in fraud detection, where features such as “merchant country” and “merchant category” may be strongly correlated. Moreover, Bayesian networks, being supervised, are susceptible to challenges from delayed labels and noisy data. Future research directions include drift-aware and cost-sensitive Bayesian models that dynamically update priors, likelihoods, and structure to adapt to concept drift; integrating representation learning

(e.g., sequence embeddings) with an online NB decision layer to mitigate independence assumption violations; and developing federated or privacy-preserving Bayesian frameworks that enable cross-institutional learning while adhering to data protection regulations, thereby improving robustness against coordinated fraud.

Hidden Markov Model

A Hidden Markov Model (HMM) is a probabilistic sequence model that represents a system as a Markov process with latent states and observable emissions. Its sequential formulation naturally lends itself to online adaptation. In credit card fraud detection, [139] trained HMMs using the Baum–Welch algorithm with observation probabilities initialized from individual spending profiles, enabling online detection via a sliding window without requiring labeled fraud signatures, though training was performed offline. [140] developed an end-to-end pipeline combining forward–backward algorithms for sequence likelihoods, Viterbi decoding for state inference, and Baum–Welch for parameter estimation, demonstrating HMM’s effectiveness for per-card behavioral modeling. Lightweight, real-time HMM-based frameworks have also been proposed [141], along with mechanisms such as one-time passwords to reduce false rejections of legitimate but atypical transactions [142]. Hybrid approaches, such as clustering transaction histories to generate discrete observation sequences for HMMs, have been applied in banking fraud detection [143].

HMMs offer several advantages for online fraud detection: they are compact yet expressive for regime-switching behavior, require few labels, can handle missing features and heterogeneous data types, and support low-latency, bounded-memory inference. However, their limitations cannot be ignored. Most applications use first-order, stationary models, which fail to capture the long-term dependencies, seasonality, and evolving tactics characteristic of real fraud. A single global HMM poorly generalizes across all entities, while per-entity models suffer from data sparsity and cold-start issues, especially for new accounts with limited history. Furthermore, standard HMMs operate in discrete time, making them sensitive to irregular inter-event intervals unless explicitly modeled. Critically, both HMMs and Naive Bayes focus on sequential patterns in isolated streams and cannot capture relational fraud patterns such as collusive rings or provider-network scams. Promising research directions include graph-coupled HMMs, time-aware Hidden Semi-Markov Models, continuous-time HMMs, and meta-learned HMMs with calibrated initialization to mitigate cold-start problems.

Other

Beyond the models previously discussed, several other classical machine learning techniques were also applied to fraud detection. For instance, fuzzy logic-based systems have been used to compute fraud scores without requiring batch retraining [144]. Random forests have demonstrated efficacy in credit card fraud detection with the potential for online adaptation through incremental learning [145]. Other notable approaches include extremely fast decision trees for streaming environments that incorporate online boosting [146] and ensemble

methods, such as AdaBoost with majority voting, that obviate the need for model retraining [147].

Despite these applications, a significant research gap persists regarding the efficient adaptation of these classical methods to address the specific challenges of online fraud detection, namely concept drift and severe data imbalance. This gap is particularly striking given that extensive research has established the foundations for adapting classical machine learning models to online or streaming contexts, including online logistic regression [148], [149], online random forests [150], [151], online fuzzy logic [152], online ensemble learning [153]–[155], online decision trees [156], online isolation forests [157], and online genetic algorithms [158]. These developments suggest substantial potential for adapting classical approaches to learn incrementally, continually, and efficiently in online fraud detection environments.

Deep Learning

With the rapid advancement of neural networks and deep learning, these techniques have seen increasing adoption in fraud detection. Unlike classical methods discussed earlier, deep learning models can automatically extract hierarchical feature representations from raw, heterogeneous data, enabling the capture of complex, nonlinear patterns often missed by traditional approaches. The following section reviews representative online deep learning architectures applied to fraud detection, including Multi-Layer Perceptrons (MLPs), Convolutional Neural Networks (CNNs), Long Short-Term Memory networks (LSTMs), Transformers, and Graph Neural Networks (GNNs).

CNN Although originally designed for image classification, Convolutional Neural Networks (CNNs) have also been applied in fraud detection. [120] employed CNNs with a feature sequencing layer for credit card fraud detection, while [159] proposed a CNN-based framework incorporating Adaptive Synthetic Sampling (ADASYN) to address class imbalance. [160] introduced a CNN–SVM hybrid model with XGBoost for enhanced classification performance. While these approaches were not explicitly designed for online training, CNN-based models can support real-time fraud detection when combined with continual learning strategies such as experience replay, mini-batch online updates, and elastic weight consolidation, as discussed in Section 2. Recent work by [161] designed a lightweight, highly parallelizable CNN architecture with efficient sequence encoding and no transformer overhead, enabling low-latency streaming inference. Similarly, [162] developed a CNN-based system explicitly for high-speed online deployment, demonstrating real-time performance and scalability in a leading fintech environment; this model has been deployed at Ant Financial (Alibaba Inc.), validating its practical applicability. [163] further proposed automated feature generation with streamlined CNN inference to reduce manual preprocessing and improve integration into real-time pipelines.

Despite advantages such as automatic feature extraction and strong performance in capturing local dependencies, CNNs have notable limitations in online fraud detection. They are

data-intensive, often requiring large volumes of labeled fraud cases, yet fraud data is typically imbalanced and labels may be delayed. Moreover, conventional CNNs struggle to capture long-range dependencies, are susceptible to concept drift, and require fixed input shapes, limiting adaptability to heterogeneous, evolving data streams.

LSTM Long Short-Term Memory (LSTM) networks, a variant of recurrent neural networks (RNNs), have been widely applied in fraud detection due to their ability to model sequential dependencies by processing data one timestep at a time, making them well-suited for online settings. [164] developed a sequence classification model that processes sequential transaction inputs incrementally, enabling real-time updates per transaction. [165], [166] incorporated attention mechanisms into LSTMs to dynamically focus on recent or salient transactions, facilitating rapid inference within streaming pipelines. [167] applied LSTMs to detect anomalies in streaming time-series data without offline batch processing, introducing proactive threshold adjustments to maintain robustness against evolving patterns. [168] proposed an explainable LSTM (X-LSTM) using a sliding-window configuration to process transactions as they occur, providing interpretable justifications for real-time alerts—critical in high-stakes monitoring environments.

While LSTMs are inherently suited to online data and can capture long-term dependencies, they present challenges for continuous deployment. They are computationally intensive, prone to catastrophic forgetting under concept drift, and susceptible to overfitting when faced with severe class imbalance or limited positive samples—issues that are particularly acute in online fraud detection.

Transformer & Large Language Models The Transformer architecture has become one of the most influential models in recent years, particularly in natural language processing, machine translation, and speech recognition, and has also been applied to fraud detection. For example, [169] proposed a Transformer-based fraud detection model integrated into a cloud-based streaming pipeline, achieving low-latency inference while maintaining accuracy. [170] utilized a GPT-style autoregressive Transformer trained in an unsupervised manner to model transaction behavior, enabling sequential, online inference as transactions arrive. [171] emphasized data preprocessing to address sparsity and feature selection to enhance correlation, allowing parallel processing of incoming data for low-latency inference, even though it was not explicitly deployed in a streaming system. [172] introduced a hybrid framework combining a Transformer encoder, multi-teacher knowledge distillation, and a symbolic Belief–Desire–Intention (BDI) reasoning layer, explicitly considering model size and inference speed for real-time deployment. The symbolic reasoning component is particularly valuable in regulated environments requiring explainability in live fraud detection.

Large Language Models (LLMs), building upon Transformer architectures, have also demonstrated potential in online fraud detection. [173] presented a real-time fraud detection system using a Retrieval-Augmented Generation (RAG)

framework that adapts to policy changes without retraining while preserving low-latency performance. [174] proposed a two-stage framework for fraud detection in conversational streams, enabling real-time adaptation to conversational drift and leveraging LLM reasoning to identify nuanced fraudulent behaviors. [175] employed multi-modal LLMs (text, reasoning, code, and vision) to assist analysts in investigating credit card alerts as they stream in, providing rapid automated support. [176] demonstrated that LLMs can detect scams by analyzing unstructured text in emails and messages, supporting real-time classification and flagging of suspicious content in streaming communications.

LLMs offer several advantages for fraud detection, including the ability to process unstructured and multi-modal data across domains such as banking, insurance, and other financial services. Their adaptability enables the creation of foundation fraud detection models that can be fine-tuned for specific tasks. Furthermore, their few-shot and zero-shot learning capabilities allow detection of novel fraud patterns with minimal or no task-specific training, reducing retraining requirements. In practice, both API-based and on-device deployments can process live data streams, enabling real-time alerts and intervention during ongoing calls, chats, or transactions, potentially stopping fraudulent activities before completion.

Graph Neural Networks Fraud rarely occurs in isolation; it often arises within networks of interconnected entities (e.g., users, accounts, devices, merchants) where relational structures are as important as individual attributes. Given the inherently relational nature of fraud, graph-based methods—particularly Graph Neural Networks (GNNs)—have emerged as a promising solution. GNNs can capture both local and global structural context, handle heterogeneous and multi-relational data, and, when combined with temporal modeling, adapt effectively to online detection scenarios. This section reviews representative applications of GNNs in fraud detection, focusing on two categories: static GNNs and dynamic GNNs with temporal modeling, also referred to as Temporal Graph Networks (TGNs).

[177] introduced a Directed Dynamic Snapshot (DDS) graph construction and a Lambda Neural Network (LNN) architecture for e-commerce checkout fraud detection. The design preserves streaming data integrity and enables efficient, real-time inference. Building on this work, [178] proposed the Batch and Real-time Inception Graph Topology (BRIGHT) framework, which employs a two-stage directed graph to restrict message passing to historical (rather than future) transactions. By separating inference into batch-based embedding computation and real-time transaction scoring, BRIGHT achieves substantially reduced latency. [179] presented an adaptive GNN incorporating a dynamic neighbor sampling strategy. The method filters noise by selecting neighbors based on behavioral similarity and leverages neighbor diversity entropy to mitigate fraudster camouflage and over-smoothing—critical capabilities for real-time systems adapting to evolving fraud behaviors. [180] proposed a heterogeneous GNN (HGNN) for credit card fraud detection that models mul-

multiple node types and applies temporal decay to prioritize recent patterns, addressing the rapid evolution of fraudulent tactics. [181] introduced FraudGT, a graph Transformer optimized for financial transaction networks, achieving low-latency inference through an efficient graph-Transformer design suitable for real-time transaction analysis. Some other interesting work includes using GNNs to combat human trafficking with weak supervision via labeling functions, implying potential of being deployed in online setting [182], [183].

Before the official introduction of TGNs, researchers have made attempts to incorporate temporal modeling in GNN-fraud detection. For instance, [184] introduced the Spatial-Temporal Attention Graph Neural Network (STA-GNN), which jointly models structural and temporal dependencies via attention, enabling incremental updates and adaptation to evolving fraud patterns in streaming environments.

Temporal Graph Neural Networks

Credit card fraud detection is one of the most suitable application domains for Temporal Graph Networks (TGNs), and the main reasons are summarized as follows:

- **Incoming data naturally forms dynamic heterogeneous graphs:** The underlying data is inherently diverse, with nodes representing entities such as card holders, merchants, IP addresses, terminal accounts, and more. Each transaction corresponds to a timestamped edge that connects a set of related entities, enabling the analysis of their temporal evolution. Furthermore, the graph structure evolves continuously as new entities and connections emerge over time.
- **Fraudulent behaviors are highly time-sensitive:** Fraudsters often carry out multiple transactions within a short time window, making timely detection critical. Certain fraud patterns occur predominantly during specific periods or seasons. In addition, the risk contribution of a transaction decreases as it recedes in time, meaning that recent transactions are typically more informative for fraud detection than older ones.
- **Support for real-time decision-making:** Conventional batch-trained GNNs require constructing the entire graph and retraining the model, which is time-consuming and unsuitable for real-time scenarios. In contrast, the message passing and memory update mechanisms in TGNs enable online, incremental updates to node representations, thereby reducing inference latency and facilitating near real-time detection.

In the following, we review several state-of-the-art TGN-based approaches for credit card fraud detection, and analyze their main architectures, novel contributions, and potential limitations.

[185] introduced CaT-GNN, a two-phase Causal Temporal Graph Neural Network for credit-card fraud detection.

- 1) **Phase 1 — Causal-Inspector:** multi-head temporal-attention scores are averaged; the bottom r_e fraction of neighbours are labeled as environment nodes, while the

rest are labeled as causal nodes. This process introduces no additional learnable parameters.

- 2) **Phase 2 — Causal-Intervener:** each environment node is linearly mixed with the top r_c causal neighbours through learnable coefficients a_i and a_j , producing a causal-mixup view that embeds counterfactual evidence without altering the original graph structure.

This approach integrates causal invariance and temporal neighbourhood partitioning into the graph modeling process, thereby improving robustness to out-of-distribution scenarios. However, it relies on the strong assumption that attention scores approximate causal effects, and empirical results show that its performance is sensitive to hyperparameter choices. Moreover, the framework operates in an offline batch-training setting rather than a strictly online, event-driven fashion.

[186] proposes GTAN, a temporal transaction graph model where each transaction is a node linked to its past transactions. A gated temporal attention module aggregates neighbor information while a learnable attribute-driven gate balances raw attributes with aggregated messages, and a masked training strategy prevents label leakage by hiding the target node’s risk during training. This design enables semi-supervised learning that propagates both features and labels through graphs. Key novelties include integrating risk embeddings as attributes, the interpretable gating mechanism, and label-leakage-safe propagation. Potential limitations are its discrete rather than continuous-time temporal modeling, dependence on partial labels, and lack of native online learning capability, which requires additional engineering efforts.

[187] applies a continuous-time event-based TGN (MPT-GNN) to an online payments dataset from MoneyPool, modeling users, cards, devices, and bank accounts as temporal graphs for fraud classification. It trains with strict chronological splits, samples the 20 most recent temporal neighbors, and compares single-event vs multi-event graph constructions, showing denser multi-event graphs yields higher F1. Its novelty lies in continuous-time memory updates, and systematic multi-graph evaluation. However, it also suffers from severe class imbalance, which leads to moderate F1, and reliance on chronological batch training rather than fully online updates.

Unlike in credit card fraud detection—where a growing body of work leverages Temporal Graph Networks or other graph neural networks capable of facilitating online learning—existing research in financial statement fraud detection and insurance fraud detection has almost no published studies employing these architectures. Most prior work in these two domains still relies on static graph representations, snapshot-based heterogeneous GNNs, or conventional machine learning models, with little emphasis on continuous-time message passing or incremental updates.

For financial statement fraud detection, datasets are often compiled from annual or quarterly filings, audit reports, or multi-year supply chain and corporate relationship data. This inherently low temporal resolution means the event sequences are sparse and irregular, with changes occurring on the scale of months or years. Such characteristics pose challenges for TGN

training, which benefits from high-frequency events to learn dynamic patterns effectively. Inference is also impacted, as the long time gaps between events reduce the utility of temporal memory updates and make continuous-time embeddings prone to staleness.

In insurance fraud detection, available datasets—such as medical or property claims—do contain temporal information, but the timelines are typically long (days to months from claim submission to resolution) and labels often arrive with significant delays due to manual investigation. Additionally, insurance claim graphs are highly heterogeneous, involving multiple entity types (policyholders, healthcare providers, procedures, drugs, adjusters) and multi-modal attributes (structured billing data, free-text case notes, medical images). These factors make straightforward TGN training more difficult: the irregular and delayed labels complicate supervised online updates, and inference pipelines must handle multi-modal integration alongside temporal message passing without exceeding latency constraints.

Nevertheless, adapting algorithms from credit card fraud detection to these two domains is feasible. For financial statement data, the TGN framework could be re-purposed to model corporate events (e.g., significant transactions, board changes, supply chain shifts) at whatever temporal granularity is available, but would require strategies to handle extreme sparsity—such as multi-scale time encoding or temporal smoothing over sparse event sequences. For insurance, the migration would need to address heterogeneous graph structures, delayed supervision, and multi-modal data fusion. In both domains, incorporating mechanisms for handling low event frequency, irregular time gaps, and concept drift will be essential to fully realize the benefits of continuous-time, online-capable graph learning.

Based on TGNN, [188] develops a real-time framework for cross-boarder-payment fraud detection that is accurate and fast. The key idea is to use domain knowledge directly into the graph model while reshaping the runtime stack for GPUs.

- 1) **Cross-boarder semantic temporal attention:** At each layer, they use a special type of attention mechanism (multi-head temporal attention). This attention is enhanced with information about currency used, the specific payment route, and the regulatory regions involved. These extra pieces of information help the model focus on potentially risky steps in payment chains that might involve multiple banks. Each point(node) in their graph combines different types of data: 64 features describing the entity itself (static), 128 features about its recent transaction behaviour (dynamic), 32 features representing time patterns (cyclic time codes), and 16 features related to risk and compliance. This gives the model a lot of useful context without using too much memory.
- 2) **Dynamic Graph Updates and Processing Flow:** The importance of connections (edge weights) between points is constantly updated during operation. This update uses four key signals: how often transactions happen, the pattern of transaction amounts, how recent

the transactions are, and the geographic risk associated with locations. This keeps the graph structure current as fraudsters change their tactics. During processing, the model first applies the attention mechanism above. Then, it processes the data through a block which includes connections that skip some layers.

- 3) **Optimized GPU Processing:** To make the model fast enough for real-time use, the authors implemented calculations using CUDA. They also used a combined LRU/FIFO caching system on the GPU to quickly access and update node and edge data (0.5ms lookup, 0.8ms update). A scheduler efficiently fed data batches to the GPU. This allowed the entire fraud detection process to finish within 100 ms, while processing over 1000 transactions per second accurately.

To conclude, our investigation indicates that graph-based approaches—particularly TGNs and other lightweight GNNs—are highly effective for online fraud detection due to their ability to capture both local and global contextual dependencies, incorporate temporal modeling, and train efficiently in semi-supervised settings with limited labeled data. Nevertheless, as deep learning methods, GNNs typically require pre-trained models for effective online deployment and incremental updates, making them vulnerable to the cold-start problem when large-scale pretraining data are unavailable. Without such pretraining, graph structures alone often yield limited predictive value. Addressing this challenge calls for research into effective pretraining strategies tailored for fraud detection. Moreover, while deep learning models generally outperform classical techniques, they demand significant computational resources, limiting applicability on edge or low-power devices. GNNs are also susceptible to adversarial structural perturbations, allowing fraudsters to manipulate graph topology to evade detection. A promising research direction is using adversarial training to robustify GNNs.

Finally, as noted in [189], no single model consistently excels across all fraud types; instead, ensembles of diverse models often enhance detection and reduce false alarms. Consequently, a promising avenue for future research is the development of hybrid models. Rather than completely abandoning classical techniques such as SVM, clustering, or isolation forests, future work should aim to create hybrid systems. Such frameworks could strategically combine the powerful representation learning of GNNs with the efficiency, interpretability, and robustness of classical methods, enabling tailored solutions that align with specific task requirements, dataset characteristics, and deployment constraints.

Table III Summarizes the strengths, weaknesses, and best-fit use cases of various methods for online fraud detection, providing a comparative guideline for model selection.

- 3) **Dataset & Evaluation:** A variety of datasets are commonly employed for fraud detection research.

For early studies on online SVMs, clustering, HMMs, and other classical techniques, the **Kaggle Credit Card Fraud Dataset** is the most widely used benchmark. It contains 284,807 European credit card transactions from 2013, in-

cluding 492 fraudulent cases (0.172% fraud rate), with 30 anonymized numerical features in addition to time and amount. Although clean, public, and highly cited, its extreme class imbalance and anonymization limit interpretability. This dataset has also been applied to CNN-based approaches. Another frequently used dataset is **PaySim**, a large-scale synthetic dataset generated from real financial logs, which captures more diverse fraud patterns and is particularly suited for sequential models such as LSTMs and Transformers. In insurance fraud detection, the Kaggle Auto Insurance Claim Dataset provides structured and interpretable features (e.g., policyholder details, claim descriptions, accident information) but is relatively small in scale. For financial statement fraud, the **Enron Email Dataset**, released after the Enron scandal, includes company financial records and insider communications, making it useful for both classical and deep learning models.

Due to the limitations of public datasets, many studies employ **proprietary datasets** from banks, payment processors (e.g., Visa, Mastercard, Chase), or insurers. These typically include anonymized transaction logs or claims data that cannot be publicly released. For instance, the **Colombia Auto Insurance Fraud Dataset** contains 48,570 insurance claim records with a 3.7% fraud ratio and 32 independent features. In financial statement fraud detection, confidential datasets from audit firms, regulators, or tax authorities (e.g., PwC, KPMG, EY, Deloitte) are also widely used, though generally inaccessible to the research community.

With the rise of graph-based methods, several graph-structured fraud datasets have emerged. The **Amazon Review Dataset** represents a heterogeneous user–product–review graph with multi-relational edges; its 2023 version includes millisecond-level timestamps for fine-grained temporal modeling [190]. The **Yelp Dataset** is another multi-relational review graph with 32 handcrafted features per node, widely used for spam detection. However, it is static and requires temporal augmentation for online learning. The **Elliptic Dataset** provides a large-scale temporal graph derived from Bitcoin transactions, containing 203,769 nodes and 234,355 edges, and is well-suited for dynamic fraud detection tasks.

A critical characteristic of all these datasets is class imbalance, reflecting real-world fraud prevalence but complicating model training by biasing predictions toward majority classes. To mitigate this, some studies generate synthetic balanced datasets by injecting fraudulent patterns into normal transaction streams, enabling controlled fraud ratios and more robust evaluation.

Some key performance metrics are as following:

- **Precision:** The proportion of predicted fraud cases that are truly fraudulent.
- **Recall:** The proportion of actual fraud cases correctly identified by the model.
- **F1 Score:** The harmonic mean of precision and recall, balancing false positives and false negatives.
- **ROC Curve:** The Receiver Operating Characteristic curve, which illustrates the trade-off between recall (true

positive rate) and the false positive rate across varying decision thresholds.

- **AUC:** The Area Under the ROC Curve, representing the probability that the model ranks a randomly chosen fraudulent case higher than a randomly chosen legitimate case.

B. Recommendation System & Personalization:

Recommendation system personalization represents one of the earliest large-scale applications of online learning, epitomized by the principle that “the more you use it, the better it becomes.” As user preferences and item popularity evolve over time, and as new users and items emerge, online learning provides a natural framework for real-time adaptation to maintain relevance. Such methods have been extensively deployed across domains including e-commerce (e.g., Amazon, Alibaba), media streaming (e.g., Netflix, Spotify, YouTube), news delivery (e.g., Google News), and social media platforms (e.g., Facebook, Twitter, TikTok).

[191] introduced a large-scale, context-aware recommender system designed to address scalability, cold-start issues, and differential service requirements in dynamic online environments. Their Adaptive Clustering Recommendation (ACR) algorithm, grounded in a contextual multi-armed bandit framework, combines offline initialization with online adaptation to both mitigate cold-start challenges and rapidly adjust to shifts in user behavior. Similarly, [192] proposed a hybrid framework integrating content-based filtering for cold-start scenarios, collaborative filtering to exploit user–item interactions, and online learning for real-time personalization. A dynamic weighting mechanism adjusts the relative contributions of these components based on current predictive accuracy, enabling incremental updates to user preference vectors and item representations without retraining from scratch. This approach enhances personalization in both initial and long-term engagement phases.

The practical impact of online learning–based personalization is further demonstrated by [193], who successfully deployed such methods at Grubhub, underscoring their effectiveness and scalability in large-scale, industry-grade recommender systems.

C. Cybersecurity and Intrusion Detection:

In network security, traffic and attack patterns drift continuously while reliable labels are scarce and delayed—especially on resource-constrained IoT/edge devices. Online machine learning enables low-latency, low-overhead adaptation as data arrive, reducing the time from novel exposure to effective defense and avoiding costly full retraining.

[194] proposed a Network Intrusion Prevention System (NIPS) combining a modified Self-Organizing Incremental Neural Network (n-SOINN) with a Support Vector Machine using a winner-takes-all (WTA) strategy. Designed for IoT and other resource-constrained environments, the system operates without signatures or rules, enabling real-time detection of both known and unknown attacks. Its architecture supports

TABLE III
COMPARISON OF TRADITIONAL AND DEEP LEARNING METHODS FOR FRAUD DETECTION IN ONLINE SETTINGS: STRENGTHS, WEAKNESSES, AND BEST-FIT USE CASES.

Method	Strengths (esp. Online)	Weaknesses (esp. Online)	Best-fit Use Cases
SVM	Max-margin generalization works well with high-dim sparse features, and supports online updates via budgeted SVM	Support vector set can grow and requires pruning, adapts slowly to drift, and suffers from poor recall under heavy imbalance	Best with solid feature engineering, moderate data scale, and when explicit threshold/cost control is needed
Clustering	Label-free, supports streaming and micro-cluster methods to track distribution shifts, provides low latency, and yields outlier scores	Hyperparameter-sensitive, provides no direct fraud/clean labels, and minorities can be swallowed by large clusters	Best suited for scenarios with severe label delay or shortage, for discovering emerging fraud rings, and for candidate generation
Naive Bayes	Uses incremental counts, is ultra-light and high-throughput, and handles drift through sliding windows or decay	Relies on a strong independence assumption that caps accuracy, and is weak on complex feature interactions and relations	Best suited as a cold-start baseline, as a lightweight first-pass filter, and when labels are sparse or delayed
HMM	Provides an explicit sequential state machine, is interpretable, fits streaming via forward recursion, and tolerates missing observations	Requires a pre-set number of states, suffers from unstable online EM and local optima, and struggles with heterogeneous features	Best suited for process-like sequences with clear phases such as transactions or claims where interpretability is needed
CNN	Good for capturing local temporal patterns, is parallel and efficient, allows easy online fine-tuning, covers longer windows through TCN, and supports 2D CNN for images or receipts	Performs weakly on very long-range dependencies, suffers from catastrophic forgetting under drift, and struggles with rare-fraud recall	Best suited for short-to-mid window transaction streams, insurance image or document verification, and multimodal feature extraction
LSTM/GRU	Models both long and short dependencies, supports stateful rolling inference that suits streams, and is somewhat robust to gradual drift	Requires truncated BPTT or replay for online training, suffers from destabilized gradients under delayed labels, and faces increasing latency with sequence length	Best suited for credit-card streams and claim processes that need per-event updates and temporal context
Transformer	Captures long-range dependencies and heterogeneous features, provides attention mechanisms that aid explanation, and supports streaming variants such as memory or sliding-window approaches	Has standard attention with $O(L^2)$ complexity, incurs costly online fine-tuning, struggles with drift adaptation, and requires tailored loss functions for imbalance	Best suited for long-context logs or transactions, multimodal fusion tasks, and scalable online adaptation using PEFT or Adapters
LLM (with RAG)	Supports zero- and few-shot text understanding for claims, reports, and chats, updates knowledge through RAG without retraining, and provides strong explanations	Is unstable as a classifier due to hallucination, cost, latency, and privacy issues, struggles to truly learn in online settings, and is weak on purely numeric anomalies	Best suited for text-heavy review or audit tasks, as a cascade after structured detectors, and for interpretable reporting
GNN	Naturally models entity relations such as card, merchant, device, and IP, enables cold-start through inductive GNNs, and supports incremental updates	Sensitive to graph construction, suffers from neighborhood explosion and staleness, involves complex online graph operations, and shows poor recall for rare fraud rings	Best suited for relationship-driven fraud such as rings, AML, or collusion, and can be combined with sequence models
TGN	Supports event-driven real-time dynamic graphs, captures fresh relations through time encoders and memory, and is sensitive to fast-evolving rings	Has memory and queue sizes that grow with scale, faces challenges in reproducibility and sampling, and requires decay strategies for drift	Best suited for high-frequency evolving interaction networks that need millisecond-level scoring

online incremental learning by continuously updating with misclassified samples via a validation and update module. Evaluated on the NSL-KDD dataset, the n-SOINN-WTA-SVM framework incrementally improved accuracy from 78.23% after initial training to 89.67% over five update rounds, while using only 27.3% of the dataset. Compared to an offline batch-trained model, it achieved higher accuracy with lower resource usage, demonstrating scalability, adaptability to evolving threats, and suitability for industrial deployment.

[195] proposed a Self-Supervised Intrusion Detection (SSID) framework that trains an IDS fully online, in parallel

with real-time detection, eliminating any need for offline (labeled or unlabeled) data and human intervention. SSID introduces a trust coefficient that combines the representativeness of previously learned traffic and the IDS’s generalization ability; using and the IDS output, it probabilistically selects and pseudo-labels incoming packets and decides when to trigger updates. Instantiated with an Auto-Associative DRNN (AADRNN)—and also validated with an MLP—the IDS reconstructs expected benign header-derived metrics and flags deviations for detection. Evaluated on Kitsune and Bot-IoT for both malicious-traffic detection and compromised-device

identification, SSID matches or surpasses the same IDS trained with offline and quasi-online (incremental/sequential) learning; on Kitsune (Mirai), it attains $\text{TPR} > 0.995$ at $\text{FPR} \approx 10^{-5}$ without any offline training. Overall, SSID offers a label-free, resource-efficient online adaptation mechanism well-suited to IoT/edge environments.

D. Stock Market Prediction & Portfolio Optimization:

The financial market—particularly the stock market—presents a highly challenging environment for machine learning due to its non-stationary, noisy, and partially observable nature, as well as the high dimensionality of market data. Real-time trading requires decisions within seconds, where errors can result in substantial financial losses. In this context, online learning offers a natural and effective framework for adapting models to rapidly evolving market conditions. Broadly, stock market prediction can be approached through *fundamental analysis* and *technical analysis*. Fundamental analysis examines intrinsic company metrics such as asset value, liabilities, and revenue, and typically demands extensive domain expertise. Technical analysis, in contrast, identifies patterns and trends from historical price movements and is more amenable to machine learning. Consequently, most existing models focus on technical analysis.

[196] developed an online Linear Discriminant Analysis (LDA) model for predicting daily stock price movement (up/down) using opening, closing, highest, and lowest prices. The model incrementally updates with new data, eliminating the need for retraining, and outperformed K-Nearest Neighbors, Decision Trees, and Artificial Neural Networks on four high-volume NASDAQ stocks—Apple, Facebook, Google, and Amazon—under both online and batch settings. Its computational efficiency further enhances its suitability for real-time prediction. [197] compared incremental learning with an *offline-online* hybrid approach. Incremental models (linear regression) adapt to each incoming tick, whereas the offline-online method (LSTM, Bi-LSTM, CNN-LSTM) retrains at the end of each trading session while predicting in real time during the day. Bi-LSTM achieved the best performance overall, with offline-online models showing greater robustness to trends and volatility, and incremental models demonstrating faster intra-day adaptation to regime shifts. These findings suggest that hybrid frameworks—combining intra-day incremental updates with overnight deep model retraining—may offer the best trade-off between short-term adaptability and long-term accuracy.

Portfolio optimization, which seeks to allocate wealth among assets (e.g., stocks, bonds, commodities) to maximize expected return while controlling risk, is another domain where online learning is highly relevant. Algorithms such as online convex optimization and multi-armed bandits enable continual adjustment of portfolio weights as new market data arrives. [198] and [199] provide comprehensive treatments of portfolio optimization within the online learning framework, linking regret minimization theory to capital growth theory in

finance. They demonstrate how strategies such as the exponential gradient, online Newton step, and passive-aggressive algorithms can be instantiated for sequential portfolio management.

In summary, online learning has demonstrated substantial utility in stock market prediction and portfolio optimization, offering both strong empirical performance in industry applications and solid theoretical guarantees [200], [201]. Despite its maturity, there remains significant potential for exploration, particularly in hybrid modeling strategies and adaptive methods for non-stationary financial environments.

E. Smart Cities, IOT Sensor System:

In smart cities and modern IoT ecosystems, sensor networks continuously generate massive amounts of real-time data for applications such as traffic management, environmental monitoring, and security surveillance. Traditional batch learning methods struggle to cope with this high-velocity, evolving data due to their reliance on static models and periodic retraining. Online machine learning offers a powerful solution by enabling models to learn and adapt incrementally from streaming data, making them highly suitable for resource-constrained devices and scenarios where timely decision-making is critical. This capability is particularly important for detecting anomalies, responding to cyber threats, and maintaining predictive accuracy in dynamic environments. The following studies exemplify how online learning approaches are being integrated into IoT and sensor systems to address these challenges.

[202] proposed an adaptive online machine learning framework for IoT botnet detection that processes streaming network traffic from resource-constrained devices. The system integrates incremental learning with ensemble methods—specifically Adaptive Random Forest (ARF) with online—and employs a prequential evaluation strategy to continuously train and predict with stopping operations. Using real IoT traffic data containing both benign flows and Mirai botnet infections, the model achieved higher accuracy and K statistics compared to state-of-the-art baselines under streaming conditions. Its ability to dynamically adjust to evolving attack patterns, combined with computational efficiency, makes it particularly suitable for real-time IoT security monitoring.

[203] proposed TinyOL, a TinyML framework enabling incremental on-device training on microcontrollers (MCUs) to address concept drift and deployment challenge in resource-constrained IoT environments. Instead of static inference-only models, TinyOL adds a trainable output layer to pre-trained neural networks, allowing real-time weight updates from streaming data without storing historical samples. Evaluated on an Arduino Nano 33 BLE Sense with an autoencoder-based anomaly detection task for a USB fan, TinyOL demonstrated two capabilities: 1. fine-tuning to align model performance with changing field conditions, restoring anomaly detection accuracy after drift, and 2. converting an unsupervised autoencoder into a supervised multi-anomaly classifier at runtime by augmenting features with encoder outputs and

reconstruction errors. While offline training on large datasets achieved slightly higher macro F1-scores, TinyOL maintained competitive accuracy with minimal memory and latency overhead, highlighting its suitability for real-time adaptation on constrained edge devices.

These studies demonstrate the effectiveness of online learning in enabling adaptive, low-latency intelligence for IoT and sensor systems. By addressing evolving conditions and concept drift under resource constraints, they highlight the potential of incremental learning to sustain high performance in dynamic smart city environments.

F. Healthcare:

In the healthcare domain, data is inherently continuous and dynamic, encompassing patient vital signs, streaming imaging data, laboratory results, and electronic health record (EHR) updates. This necessitates real-time adaptation for tasks such as treatment adjustment, outbreak monitoring, and personalized care.

[204] provide a comprehensive discussion of the opportunities and challenges in applying online learning to clinical medicine. Analogous to recommendation systems, online learning in healthcare can update diagnostic, prognostic, or treatment models as patient data accumulates, thereby improving predictive accuracy, enhancing adaptability to shifting patient populations, and reducing the dependence on massive static datasets. However, challenges include catastrophic forgetting, regulatory and ethical constraints, trust and adoption barriers, and strict privacy requirements.

To address model degradation in multi-institutional settings, [205] propose Weight Uncertainty Propagation and Episodic Representation Replay (WUPERR), a privacy-preserving continual learning framework. WUPERR enables sequential learning from different hospital cohorts without sharing raw patient data, mitigating catastrophic forgetting by freezing the first network layer post-initial training and storing only learned representations. Similarly, [206] investigate the feasibility of training a single general-purpose medical imaging model capable of sequentially learning new tasks without substantial performance loss on prior ones. Their results indicate that replay-based methods (e.g., REMIND) consistently outperform regularization-only approaches (e.g., Memory Aware Synapses). [207] explore the application of reinforcement learning (RL) in online healthcare settings, formulating problems as Markov Decision Processes. This enables optimization of sequential treatment plans for chronic conditions, dynamic drug dosing based on real-time patient response, and adaptive diagnostic strategies during image acquisition.

In summary, online learning holds significant promise for healthcare applications by enabling adaptive, data-driven decision-making in dynamic clinical environments. While its potential to improve diagnostic accuracy, personalize treatment, and enhance operational efficiency is clear, practical deployment must address challenges such as catastrophic forgetting, regulatory compliance, data privacy, and trust. Building robust foundational models is equally critical, as they

provide a strong base for continuous adaptation and task generalization. Future research should prioritize the development of privacy-preserving, interpretable, and scalable online learning frameworks tailored to the unique constraints of healthcare systems.

V. CHALLENGES & PROMISING SOLUTION

Online learning provides a powerful framework for modeling sequential data across diverse applications. However, several critical challenges remain unresolved. This section highlights three of the most pressing issues in fraud detection.

Concept drift. A major challenge is concept drift, including both natural and adversarial forms. Natural drift arises from seasonal or behavioral changes, while adversarial drift occurs when fraudsters deliberately adapt their strategies. Existing approaches—such as sliding windows, selective memory restoration, or incremental frameworks [208]–[210]—offer partial solutions, but no unified protocol exists for reliably detecting, attributing, and responding to different types of drift.

Delayed and biased labels. Another fundamental issue is label delay. In fraud detection, labels often arrive weeks later and only for cases that have been manually reviewed (e.g., customer disputes), inducing severe feedback bias. Fraudulent cases that escape detection never appear in the training data. This undermines theoretical guarantees of online convex optimization, which typically assumes immediate feedback. In practice, only 1–5% of cases are labeled, often with long delays, and labels are policy-biased since analysts selectively review cases. These challenges demand methods that integrate positive-unlabeled learning, survival analysis, or bias-corrected updates tailored to online fraud detection.

Class imbalance. Fraud is exceedingly rare, typically representing less than 1% of transactions. This extreme imbalance renders accuracy meaningless and can distort metrics such as AUC-ROC. Online algorithms such as Perceptron or Online Gradient Descent optimize symmetric loss functions, which underweight the minority class. While synthetic oversampling has been explored, it fails to capture the complexity of real-world fraud. Three directions are particularly important: (1) adaptive thresholding under analyst capacity constraints, (2) memory-efficient replay or exemplar strategies to prevent minority-class forgetting, and (3) adoption of imbalance-aware metrics such as AUC-PR as primary benchmarks.

Beyond these core issues, additional challenges warrant attention. These include stability of training under dynamic graph structures and multi-entity reasoning, deployment constraints such as memory, latency, and energy budgets, as well as ethical concerns encompassing privacy, fairness, and governance. Addressing these challenges is essential for advancing the reliability and real-world applicability of online fraud detection systems.

A promising avenue for addressing these challenges lies in the emerging field of quantum computing. Recent efforts illustrate its potential: [211] proposed a Quantum Graph Neural Network (QGNN) based on variational quantum circuits for financial fraud detection, achieving superior accuracy and

robustness compared to classical GNNs on the Kaggle dataset. Similarly, [212] argued that quantum computing, by exploiting qubits and quantum mechanics, can transform finance through faster and more accurate fraud detection, trading optimization, and risk profiling. In addition, [213] introduced a hybrid quantum–classical framework in which quantum feature selection enhances classical machine learning models, yielding improvements in both accuracy and efficiency. Collectively, these studies suggest that quantum graph neural networks, quantum walks, and hybrid quantum–classical architectures hold promise for modeling fraud rings, detecting anomalies in large-scale networks, and enabling low-latency decision-making.

Quantum computing also offers potential solutions to the three core challenges of online fraud detection. For concept drift, quantum kernels and distance measures may capture distributional shifts in exponentially larger feature spaces, facilitating model adaptation without catastrophic forgetting. In the context of severe class imbalance, amplitude amplification can reweight rare fraud instances more effectively than classical oversampling, while variational quantum circuits can directly optimize cost-sensitive objectives to preserve precision under skewed distributions. For delayed labels, quantum systems’ ability to maintain superpositions of hypotheses and update them as evidence arrives suggests new mechanisms for handling uncertain or lagged feedback. Moreover, quantum bandit formulations may accelerate exploration–exploitation trade-offs under delayed rewards in reinforcement learning settings.

While current noisy intermediate-scale quantum (NISQ) devices are not yet capable of real-time deployment, the theoretical advantages of quantum computing underscore its promise as a long-term pathway for overcoming the intrinsic limitations of classical online learning in fraud detection.

VI. CONCLUSION

Online machine learning has emerged as a crucial paradigm for addressing dynamic, large-scale, and high-stakes environments where data arrive in a continual stream and immediate adaptation is essential. This review has surveyed the theoretical foundations of online learning, including regret minimization and classical algorithms such as Perceptron, Winnow, Online Gradient Descent, and Passive–Aggressive methods, as well as their extensions to kernel-based and deep learning frameworks. We have further highlighted the unique suitability of online learning for fraud detection, where adversaries adapt rapidly, data distributions shift over time, and timely decision-making is critical.

Despite significant progress, substantial challenges remain. In real-world fraud detection systems, severe class imbalance and delayed or selective labels continue to hinder robust model training. Concept drift, adversarial manipulation, and the stability–plasticity trade-off complicate deployment at scale. Moreover, practical constraints—such as latency, memory budgets, and the need for calibrated, fair, and interpretable pre-

dictions—underscore the gap between theoretical algorithms and production-ready systems.

Future research should focus on delay-aware and imbalance-sensitive learning strategies, drift-robust and adversarially resilient models, and resource-constrained algorithms with formal guarantees. Equally important is the development of realistic benchmarks and evaluation protocols that capture the operational constraints of fraud detection, including cost-sensitive metrics, analyst workload, and regulatory considerations. Beyond fraud, the principles discussed here extend naturally to healthcare, cybersecurity, recommender systems, and financial forecasting, where continuous adaptation is indispensable.

In conclusion, online machine learning provides a powerful foundation for building adaptive and resilient intelligent systems. By advancing methods that explicitly address delayed feedback, imbalance, drift, and deployment constraints, the community can bridge the gap between elegant theory and reliable practice, enabling trustworthy AI systems for fraud detection and beyond.

REFERENCES

- [1] D. Wu, S. Zhuo, Y. Wang, Z. Chen, and Y. He, “Online semi-supervised learning with mix-typed streaming features,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 4720–4728.
- [2] Y.-Y. Shen, Y.-M. Zhang, X.-Y. Zhang, and C.-L. Liu, “Online semi-supervised learning with learning vector quantization,” *Neurocomputing*, vol. 399, pp. 467–478, 2020.
- [3] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, “icarl: Incremental classifier and representation learning,” in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2017, pp. 2001–2010.
- [4] H. Shin, J. K. Lee, J. Kim, and J. Kim, “Continual learning with deep generative replay,” *Advances in neural information processing systems*, vol. 30, 2017.
- [5] D. Lopez-Paz and M. Ranzato, “Gradient episodic memory for continual learning,” *Advances in neural information processing systems*, vol. 30, 2017.
- [6] T. L. Hayes, N. D. Cahill, and C. Kanan, “Memory efficient experience replay for streaming learning,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 9769–9776.
- [7] R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio, “Gradient based sample selection for online continual learning,” *Advances in neural information processing systems*, vol. 32, 2019.
- [8] R. Aljundi, E. Belilovsky, T. Tuytelaars, L. Charlin, M. Caccia, M. Lin, and L. Page-Caccia, “Online continual learning with maximal interfered retrieval,” *Advances in neural information processing systems*, vol. 32, 2019.
- [9] S. Min, “Rethinking data use in large language models,” Ph.D. dissertation, University of Washington, 2024.
- [10] G. Liang, Z. Chen, S. Su, S. Zhang, and Y. Zhang, “A masking, linkage and guidance framework for online class incremental learning,” *Pattern Recognition*, vol. 160, p. 111185, 2025.
- [11] E. Lin, W. Luo, W. Jia, Y. Chen, and S. Yang, “Online continual learning via spiking neural networks with sleep enhanced latent replay,” *arXiv preprint arXiv:2507.02901*, 2025.
- [12] M. Perini, G. Ramponi, P. Carbone, and V. Kalavri, “Learning on streaming graphs with experience replay,” in *Proceedings of the 37th ACM/SIGAPP symposium on applied computing*, 2022, pp. 470–478.
- [13] S. Kapturowski, G. Ostrovski, J. Quan, R. Munos, and W. Dabney, “Recurrent experience replay in distributed reinforcement learning,” in *International conference on learning representations*, 2018.
- [14] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” *arXiv preprint arXiv:1511.05952*, 2015.

- [15] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight experience replay," *Advances in neural information processing systems*, vol. 30, 2017.
- [16] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell, "Progressive neural networks," *arXiv preprint arXiv:1606.04671*, 2016.
- [17] C. Fernando, D. Banarse, C. Blundell, Y. Zwols, D. Ha, A. A. Rusu, A. Pritzel, and D. Wierstra, "Pathnet: Evolution channels gradient descent in super neural networks," *arXiv preprint arXiv:1701.08734*, 2017.
- [18] A. Mallya, D. Davis, and S. Lazebnik, "Piggyback: Adapting a single network to multiple tasks by learning to mask weights," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 67–82.
- [19] A. Mallya and S. Lazebnik, "Packnet: Adding multiple tasks to a single network by iterative pruning," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2018, pp. 7765–7773.
- [20] M. Wortsman, V. Ramanujan, R. Liu, A. Kembhavi, M. Rastegari, J. Yosinski, and A. Farhadi, "Supermasks in superposition," *Advances in neural information processing systems*, vol. 33, pp. 15 173–15 184, 2020.
- [21] J. Von Oswald, C. Henning, B. F. Grewe, and J. Sacramento, "Continual learning with hypernetworks," *arXiv preprint arXiv:1906.00695*, 2019.
- [22] J. Yoon, S. Kim, E. Yang, and S. J. Hwang, "Scalable and order-robust continual learning with additive parameter decomposition," *arXiv preprint arXiv:1902.09432*, 2019.
- [23] S. Golkar, M. Kagan, and K. Cho, "Continual learning via neural pruning," *arXiv preprint arXiv:1903.04476*, 2019.
- [24] P. Zhang, Y. Yan, C. Li, S. Wang, X. Xie, G. Song, and S. Kim, "Continual learning on dynamic graphs via parameter isolation," in *Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval*, 2023, pp. 601–611.
- [25] J. Yoon, E. Yang, J. Lee, and S. J. Hwang, "Lifelong learning with dynamically expandable networks," *arXiv preprint arXiv:1708.01547*, 2017.
- [26] D. Sahoo, Q. Pham, J. Lu, and S. C. Hoi, "Online deep learning: Learning deep neural networks on the fly," *arXiv preprint arXiv:1711.03705*, 2017.
- [27] C. Cortes, X. Gonzalvo, V. Kuznetsov, M. Mohri, and S. Yang, "Adanet: Adaptive structural learning of artificial neural networks," in *International conference on machine learning*. PMLR, 2017, pp. 874–883.
- [28] S. Yan, J. Xie, and X. He, "Der: Dynamically expandable representation for class incremental learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 3014–3023.
- [29] K. Luong and M. Thielscher, "Hierarchically gated experts for efficient online continual learning," *arXiv preprint arXiv:2412.17188*, 2024.
- [30] S. Cai, Z. Xu, Z. Huang, Y. Chen, and C.-C. J. Kuo, "Enhancing cnn incremental learning capability with an expanded network," in *2018 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2018, pp. 1–6.
- [31] K. Javed and M. White, "Meta-learning representations for continual learning," *Advances in neural information processing systems*, vol. 32, 2019.
- [32] G. Gupta, K. Yadav, and L. Paull, "Look-ahead meta learning for continual learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 11 588–11 598, 2020.
- [33] C. Finn, A. Rajeswaran, S. Kakade, and S. Levine, "Online meta-learning," in *International conference on machine learning*. PMLR, 2019, pp. 1920–1930.
- [34] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska et al., "Overcoming catastrophic forgetting in neural networks," *Proceedings of the national academy of sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [35] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *International conference on machine learning*. PMLR, 2017, pp. 3987–3995.
- [36] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 139–154.
- [37] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 532–547.
- [38] H. Ahn, S. Cha, D. Lee, and T. Moon, "Uncertainty-based continual learning with adaptive regularization," *Advances in neural information processing systems*, vol. 32, 2019.
- [39] L. Korycki and B. Krawczyk, "Class-incremental experience replay for continual learning under concept drift," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 3649–3658.
- [40] X. Liu, C. Wu, M. Menta, L. Herranz, B. Raducanu, A. D. Bagdanov, S. Jui, and J. v. de Weijer, "Generative feature replay for class-incremental learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, 2020, pp. 226–227.
- [41] G.-M. Park, S.-M. Yoo, and J.-H. Kim, "Convolutional neural network with developmental memory for continual learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 6, pp. 2691–2705, 2020.
- [42] T. L. Hayes and C. Kanan, "Lifelong machine learning with deep streaming linear discriminant analysis," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, 2020, pp. 220–221.
- [43] J. Park, M. Kang, and B. Han, "Class-incremental learning for action recognition in videos," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 13 698–13 707.
- [44] D. Roy, P. Panda, and K. Roy, "Tree-cnn: A hierarchical deep convolutional neural network for incremental learning," 2019. [Online]. Available: <https://arxiv.org/abs/1802.05800>
- [45] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.
- [46] T. Ergen and S. S. Kozat, "Efficient online learning algorithms based on lstm neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3772–3783, 2018.
- [47] T. Ergen, A. H. Mirza, and S. S. Kozat, "Energy-efficient lstm networks for online learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 8, pp. 3114–3126, 2020.
- [48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [49] N. Moritz, T. Hori, and J. Le, "Streaming automatic speech recognition with the transformer model," in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 6074–6078.
- [50] X. Chen, Y. Wu, Z. Wang, S. Liu, and J. Li, "Developing real-time streaming transformer transducer for speech recognition on large-scale dataset," 2021. [Online]. Available: <https://arxiv.org/abs/2010.11395>
- [51] A. Douillard, A. Ramé, G. Couairon, and M. Cord, "Dytox: Transformers for continual learning with dynamic token expansion," 2022. [Online]. Available: <https://arxiv.org/abs/2111.11326>
- [52] Z. Wang, L. Liu, Y. Duan, Y. Kong, and D. Tao, "Continual learning with lifelong vision transformer," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 171–181.
- [53] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning: going beyond euclidean data," *IEEE Signal Processing Magazine*, vol. 34, no. 4, pp. 18–42, 2017.
- [54] P. Battaglia, R. Pascanu, M. Lai, D. Jimenez Rezende et al., "Interaction networks for learning about objects, relations and physics," *Advances in neural information processing systems*, vol. 29, 2016.
- [55] L. Galke, I. Vagliano, and A. Scherp, "Can graph neural networks go online"? an analysis of pretraining and inference," *arXiv preprint arXiv:1905.06018*, 2019.
- [56] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," *arXiv preprint arXiv:2006.10637*, 2020.
- [57] R. Hisano, "Semi-supervised graph embedding approach to dynamic link prediction," in *International workshop on complex networks*. Springer, 2018, pp. 109–121.
- [58] W. Yu, W. Cheng, C. C. Aggarwal, H. Chen, and W. Wang, "Link prediction with spatial and temporal consistency in dynamic networks," in *IJCAI*, 2017, pp. 3343–3349.

- [59] J. Zhu, Q. Xie, and E. J. Chin, "A hybrid time-series link prediction framework for large social network," in *International Conference on Database and Expert Systems Applications*. Springer, 2012, pp. 345–359.
- [60] L. Yao, L. Wang, L. Pan, and K. Yao, "Link prediction based on common-neighbors for dynamic social network," *Procedia Computer Science*, vol. 83, pp. 82–89, 2016.
- [61] Y. Seo, M. Defferrard, P. Vandergheynst, and X. Bresson, "Structured sequence modeling with graph convolutional recurrent networks," in *International conference on neural information processing*. Springer, 2018, pp. 362–373.
- [62] F. Manessi, A. Rozza, and M. Manzo, "Dynamic graph convolutional networks," *Pattern Recognition*, vol. 97, p. 107000, 2020.
- [63] J. Chen, X. Wang, and X. Xu, "Gc-lstm: Graph convolution embedded lstm for dynamic network link prediction," *Applied Intelligence*, vol. 52, no. 7, pp. 7513–7528, 2022.
- [64] L. Zhou, Y. Yang, X. Ren, F. Wu, and Y. Zhuang, "Dynamic network embedding by modeling triadic closure process," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
- [65] U. Singer, I. Guy, and K. Radinsky, "Node embedding over temporal graphs," *arXiv preprint arXiv:1903.08889*, 2019.
- [66] P. Goyal, N. Kamra, X. He, and Y. Liu, "Dyngem: Deep embedding method for dynamic graphs," *arXiv preprint arXiv:1805.11273*, 2018.
- [67] Y. Pei, J. Zhang, G. Fletcher, and M. Pechenizkiy, "Node classification in dynamic social networks," *Proceedings of AALTD*, p. 54, 2016.
- [68] S. Mahdavi, S. Khoshraftar, and A. An, "dynnode2vec: Scalable dynamic network embedding," in *2018 IEEE international conference on big data (Big Data)*. IEEE, 2018, pp. 3762–3765.
- [69] D. M. Dunlavy, T. G. Kolda, and E. Acar, "Temporal link prediction using matrix and tensor factorizations," *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 5, no. 2, pp. 1–27, 2011.
- [70] Y. Xin, Z.-Q. Xie, and J. Yang, "An adaptive random walk sampling method on dynamic community detection," *Expert Systems with Applications*, vol. 58, pp. 10–19, 2016.
- [71] S. De Winter, T. Decuyper, S. Mitrović, B. Baesens, and J. De Weerd, "Combining temporal aspects of dynamic networks with node2vec for a more efficient dynamic link prediction," in *2018 IEEE/ACM international conference on advances in social networks analysis and mining (ASONAM)*. IEEE, 2018, pp. 1234–1241.
- [72] J. Zhang, X. Shi, J. Xie, H. Ma, I. King, and D.-Y. Yeung, "Gaan: Gated attention networks for learning on large and spatiotemporal graphs," *arXiv preprint arXiv:1803.07294*, 2018.
- [73] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," *arXiv preprint arXiv:1707.01926*, 2017.
- [74] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. Schardl, and C. Leiserson, "Evolvegc: Evolving graph convolutional networks for dynamic graphs," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 5363–5370.
- [75] W. Yu, W. Cheng, C. C. Aggarwal, K. Zhang, H. Chen, and W. Wang, "Netwalk: A flexible deep embedding approach for anomaly detection in dynamic networks," in *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018, pp. 2672–2681.
- [76] B. Yu, M. Li, J. Zhang, and Z. Zhu, "3d graph convolutional networks with temporal graphs: A spatial information free framework for traffic forecasting," *arXiv preprint arXiv:1903.00919*, 2019.
- [77] A. Sankar, Y. Wu, L. Gou, W. Zhang, and H. Yang, "Dysat: Deep neural representation learning on dynamic graphs via self-attention networks," in *Proceedings of the 13th international conference on web search and data mining*, 2020, pp. 519–527.
- [78] R. Goel, S. M. Kazemi, M. Brubaker, and P. Poupert, "Diachronic embedding for temporal knowledge graph completion," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 04, 2020, pp. 3988–3995.
- [79] C. Xu, M. Nayyeri, F. Alkhoury, H. Yazdi, and J. Lehmann, "Temporal knowledge graph completion based on time series gaussian embedding," in *International Semantic Web Conference*. Springer, 2020, pp. 654–671.
- [80] S. S. Dasgupta, S. N. Ray, and P. Talukdar, "Hyte: Hyperplane-based temporally aware knowledge graph embedding," in *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, pp. 2001–2011.
- [81] A. García-Durán, S. Dumančić, and M. Niepert, "Learning sequence encoders for temporal knowledge graph completion," *arXiv preprint arXiv:1809.03202*, 2018.
- [82] G. H. Nguyen, J. B. Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim, "Continuous-time dynamic network embeddings," in *Companion proceedings of the the web conference 2018*, 2018, pp. 969–976.
- [83] —, "Dynamic network embeddings: From random walks to temporal random walks," in *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018, pp. 1085–1092.
- [84] N. Bastas, T. Semertzidis, A. Axenopoulos, and P. Daras, "evolve2vec: Learning network representations using temporal unfolding," in *International conference on multimedia modeling*. Springer, 2018, pp. 447–458.
- [85] R. Trivedi, H. Dai, Y. Wang, and L. Song, "Know-evolve: Deep temporal reasoning for dynamic knowledge graphs," in *international conference on machine learning*. PMLR, 2017, pp. 3462–3471.
- [86] Y. Ma, V. Tresp, and E. A. Daxberger, "Embedding models for episodic knowledge graphs," *Journal of Web Semantics*, vol. 59, p. 100490, 2019.
- [87] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, "Dyrep: Learning representations over dynamic graphs," in *International conference on learning representations*, 2019.
- [88] S. Kumar, X. Zhang, and J. Leskovec, "Predicting dynamic embedding trajectory in temporal interaction networks," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 1269–1278.
- [89] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," *arXiv preprint arXiv:2002.07962*, 2020.
- [90] M. D. Akers and J. L. Gissel, "What is fraud and who is responsible?" *Journal of Forensic Accounting*, 2006.
- [91] A. Reurink, "Financial fraud: A literature review," *Contemporary topics in finance: A collection of literature surveys*, pp. 79–115, 2019.
- [92] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection and concept-drift adaptation with delayed supervised information," in *2015 international joint conference on Neural networks (IJCNN)*. IEEE, 2015, pp. 1–8.
- [93] T. Fawcett and F. Provost, "Adaptive fraud detection," *Data mining and knowledge discovery*, vol. 1, no. 3, pp. 291–316, 1997.
- [94] H. O. Bello, A. B. Ige, and M. N. Ameyaw, "Adaptive machine learning models: Concepts for real-time financial fraud prevention in dynamic environments," *World Journal of Advanced Engineering Technology and Sciences*, vol. 12, no. 02, pp. 021–034, 2024.
- [95] S. B. E. Raj and A. A. Portia, "Analysis on credit card fraud detection methods," in *2011 International Conference on Computer, Communication and Electrical Technology (ICCCET)*. IEEE, 2011, pp. 152–156.
- [96] Z. Rezaee, "Causes, consequences, and deterrence of financial statement fraud," *Critical perspectives on Accounting*, vol. 16, no. 3, pp. 277–298, 2005.
- [97] C. E. Hogan, Z. Rezaee, R. A. Riley Jr, and U. K. Velury, "Financial statement fraud: Insights from the academic literature," *Auditing: A Journal of Practice & Theory*, vol. 27, no. 2, pp. 231–252, 2008.
- [98] R. A. Derrig, "Insurance fraud," *Journal of Risk and Insurance*, vol. 69, no. 3, pp. 271–287, 2002.
- [99] S. Viaene and G. Dedene, "Insurance fraud: Issues and challenges," *The Geneva Papers on Risk and Insurance-Issues and Practice*, vol. 29, no. 2, pp. 313–333, 2004.
- [100] S. W. Buell, "What is securities fraud," *Duke LJ*, vol. 61, p. 511, 2011.
- [101] M. Levi and P. Reuter, "Money laundering," *Crime and justice*, vol. 34, no. 1, pp. 289–375, 2006.
- [102] A. T. Carswell and D. C. Bachtel, "Mortgage fraud: A risk factor analysis of affected communities," *Crime, law and social change*, vol. 52, no. 4, pp. 347–364, 2009.
- [103] M. Kutera, "Cryptocurrencies as a subject of financial fraud," *Journal of Entrepreneurship, Management and Innovation*, vol. 18, no. 4, pp. 45–77, 2022.
- [104] L. Shelley, "Human trafficking," *A global perspective*, 2010.
- [105] H. Weng, Z. Li, S. Ji, C. Chu, H. Lu, T. Du, and Q. He, "Online e-commerce fraud: a large-scale detection and analysis," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1435–1440.

- [106] F. H. Glancy and S. B. Yadav, "A computational model for financial reporting fraud detection," *Decision support systems*, vol. 50, no. 3, pp. 595–601, 2011.
- [107] C. S. Throckmorton, W. J. Mayew, M. Venkatachalam, and L. M. Collins, "Financial fraud detection using vocal, linguistic and financial cues," *Decision Support Systems*, vol. 74, pp. 78–87, 2015.
- [108] M. Kirlidog and C. Asuk, "A fraud detection approach with data mining in health insurance," *Procedia-Social and Behavioral Sciences*, vol. 62, pp. 989–994, 2012.
- [109] P. Hajek and R. Henriques, "Mining corporate annual reports for intelligent detection of financial statement fraud—a comparative study of machine learning methods," *Knowledge-Based Systems*, vol. 128, pp. 139–152, 2017.
- [110] S. Ertekin, L. Bottou, and C. L. Giles, "Nonconvex online support vector machines," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 2, pp. 368–381, 2010.
- [111] X. Miao, Y. Liu, H. Zhao, and C. Li, "Distributed online one-class support vector machine for anomaly detection over networks," *IEEE transactions on cybernetics*, vol. 49, no. 4, pp. 1475–1488, 2018.
- [112] N. K. Gyamfi and J.-D. Abdulai, "Bank fraud detection using support vector machine," in *2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON)*. IEEE, 2018, pp. 37–41.
- [113] I. Rajak and K. J. Mathai, "Intelligent fraudulent detection system based svm and optimized by danger theory," in *2015 International Conference on Computer, Communication and Control (IC4)*. IEEE, 2015, pp. 1–4.
- [114] J. R. D. Kho and L. A. Veal, "Credit card fraud detection based on transaction behavior," in *TENCON 2017-2017 IEEE Region 10 Conference*. IEEE, 2017, pp. 1880–884.
- [115] M. Jeragh and M. AlSulaimi, "Combining auto encoders and one class support vectors machine for fraudulent credit card transactions detection," in *2018 Second World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4)*. IEEE, 2018, pp. 178–184.
- [116] S. Subudhi and S. Panigrahi, "Effect of class imbalance in detecting automobile insurance fraud," in *2018 2nd international conference on data science and business analytics (ICDSBA)*. IEEE, 2018, pp. 528–531.
- [117] X. Li and S. Ying, "Lib-svms detection model of regulating-profits financial statement fraud using data of chinese listed companies," in *2010 International Conference on E-Product E-Service and E-Entertainment*. IEEE, 2010, pp. 1–4.
- [118] V. Mareeswari and G. Gunasekaran, "Prevention of credit card fraud detection based on hsvm," in *2016 International conference on information communication and embedded systems (ICICES)*. IEEE, 2016, pp. 1–4.
- [119] S.-Y. Huang, R.-H. Tsai, and F. Yu, "Topological pattern discovery and feature extraction for fraudulent financial reporting," *Expert systems with applications*, vol. 41, no. 9, pp. 4360–4372, 2014.
- [120] Z. Zhang, X. Zhou, X. Zhang, L. Wang, and P. Wang, "A model based on convolutional neural network for online transaction fraud detection," *Security and Communication Networks*, vol. 2018, no. 1, p. 5680264, 2018.
- [121] I. Benchaji, S. Douzi, and B. El Ouahidi, "Using genetic algorithm to improve classification of imbalanced datasets for credit card fraud detection," in *International Conference on Advanced Information Technology, Services and Systems*. Springer, 2018, pp. 220–229.
- [122] M. Anbarasi and S. Dhiyya, "Fraud detection using outlier predictor in health insurance data," in *2017 International Conference on Information Communication and Embedded Systems (ICICES)*. IEEE, 2017, pp. 1–6.
- [123] W. Barbakh and C. Fyfe, "Online clustering algorithms," *International journal of neural systems*, vol. 18, no. 03, pp. 185–194, 2008.
- [124] A. Choromanska and C. Monteleoni, "Online clustering with experts," in *Artificial Intelligence and Statistics*. PMLR, 2012, pp. 227–235.
- [125] V. Cohen-Addad, B. Guedj, V. Kanade, and G. Rom, "Online k-means clustering," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2021, pp. 1126–1134.
- [126] R. Hyde, P. Angelov, and A. R. MacKenzie, "Fully online clustering of evolving data streams into arbitrarily shaped clusters," *Information Sciences*, vol. 382, pp. 96–114, 2017.
- [127] A. Mollaoğlu, G. Baltaoğlu, E. Çakır, and M. S. Aktaş, "Fraud detection on streaming customer behavior data with unsupervised learning methods," in *2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*. IEEE, 2021, pp. 1–6.
- [128] R. A. Ariyaluran Habeeb, F. Nasaruddin, A. Gani, M. A. Amanullah, I. Abaker Targio Hashem, E. Ahmed, and M. Imran, "Clustering-based real-time anomaly detection—a breakthrough in big data technologies," *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 8, p. e3647, 2022.
- [129] S. Maes, K. Tuyls, B. Vanschoenwinkel, and B. Manderick, "Credit card fraud detection using bayesian and neural networks," in *Proceedings of the 1st international naio congress on neuro fuzzy technologies*, vol. 261, 2002, p. 270.
- [130] S. Vaeae, G. Dedene, and R. A. Derrig, "Auto claim fraud detection using bayesian learning neural networks," *Expert systems with applications*, vol. 29, no. 3, pp. 653–666, 2005.
- [131] E. W. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature," *Decision support systems*, vol. 50, no. 3, pp. 559–569, 2011.
- [132] Q. Deng, "Detection of fraudulent financial statements based on naïve bayes classifier," in *2010 5th international conference on computer science & education*. IEEE, 2010, pp. 1032–1035.
- [133] S. Panigrahi, A. Kundu, S. Sural, and A. K. Majumdar, "Credit card fraud detection: A fusion approach using dempster–shafer theory and bayesian learning," *Information Fusion*, vol. 10, no. 4, pp. 354–363, 2009.
- [134] A. G. de Sá, A. C. Pereira, and G. L. Pappa, "A customized classification algorithm for credit card fraud detection," *Engineering Applications of Artificial Intelligence*, vol. 72, pp. 21–29, 2018.
- [135] J. O. Awoyemi, A. O. Adetunmbi, and S. A. Oluwadare, "Credit card fraud detection using machine learning techniques: A comparative analysis," in *2017 international conference on computing networking and informatics (ICCNi)*. IEEE, 2017, pp. 1–9.
- [136] R. A. Bauder, T. M. Khoshgoftaar, A. Richter, and M. Herland, "Predicting medical provider specialties to detect anomalous insurance claims," in *2016 IEEE 28th international conference on tools with artificial intelligence (ICTAI)*. IEEE, 2016, pp. 784–790.
- [137] F. Gumus, C. O. Sakar, Z. Erdem, and O. Kursun, "Online naïve bayes classification for network intrusion detection," in *2014 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM 2014)*. IEEE, 2014, pp. 670–674.
- [138] S.-H. Li, D. C. Yen, W.-H. Lu, and C. Wang, "Identifying the signs of fraudulent accounts using data mining techniques," *Computers in Human Behavior*, vol. 28, no. 3, pp. 1002–1013, 2012.
- [139] A. Srivastava, A. Kundu, S. Sural, and A. Majumdar, "Credit card fraud detection using hidden markov model," *IEEE Transactions on dependable and secure computing*, vol. 5, no. 1, pp. 37–48, 2008.
- [140] D. Iyer, A. Mohanpurkar, S. Janardhan, D. Rathod, and A. Sardeshmukh, "Credit card fraud detection using hidden markov model," in *2011 World Congress on Information and Communication Technologies*. IEEE, 2011, pp. 1062–1066.
- [141] V. Bhusari and S. Patil, "Study of hidden markov model in credit card fraudulent detection," *International Journal of Computer Applications*, vol. 20, no. 5, pp. 33–36, 2011.
- [142] S. S. Mhamane and L. Lobo, "Internet banking fraud detection using hmm," in *2012 Third International Conference on Computing, Communication and Networking Technologies (ICCCNT'12)*. IEEE, 2012, pp. 1–4.
- [143] X. Wang, H. Wu, and Z. Yi, "Research on bank anti-fraud model based on k-means and hidden markov model," in *2018 IEEE 3rd International Conference on Image, Vision and Computing (ICIVC)*. IEEE, 2018, pp. 780–784.
- [144] G. Charizanos, H. Demirhan, and D. İçen, "An online fuzzy fraud detection framework for credit card transactions," *Expert Systems with Applications*, vol. 252, p. 124127, 2024.
- [145] S. Xuan, G. Liu, Z. Li, L. Zheng, S. Wang, and C. Jiang, "Random forest for credit card fraud detection," in *2018 IEEE 15th international conference on networking, sensing and control (ICNSC)*. IEEE, 2018, pp. 1–6.
- [146] A. A. Khine and H. W. Khin, "Credit card fraud detection using online boosting with extremely fast decision tree," in *2020 IEEE conference on computer applications (ICCA)*. IEEE, 2020, pp. 1–4.

- [147] K. Randhawa, C. K. Loo, M. Seera, C. P. Lim, and A. K. Nandi, "Credit card fraud detection using adaboost and majority voting," *IEEE access*, vol. 6, pp. 14 277–14 284, 2018.
- [148] H. B. McMahan and M. Streeter, "Open problem: Better bounds for online logistic regression," in *Conference on Learning Theory. JMLR Workshop and Conference Proceedings*, 2012, pp. 44–1.
- [149] E. Hazan, T. Koren, and K. Y. Levy, "Logistic regression: Tight bounds for stochastic and online optimization," in *Conference on Learning Theory*. PMLR, 2014, pp. 197–209.
- [150] B. Lakshminarayanan, D. M. Roy, and Y. W. Teh, "Mondrian forests: Efficient online random forests," *Advances in neural information processing systems*, vol. 27, 2014.
- [151] Y. Zhong, H. Yang, Y. Zhang, and P. Li, "Online random forests regression with memories," *Knowledge-Based Systems*, vol. 201, p. 106058, 2020.
- [152] M. Arguedas, F. Xhafa, L. Casillas, T. Daradoumis, A. Peña, and S. Caballé, "A model for providing emotion awareness and feedback using fuzzy logic in online learning," *Soft Computing*, vol. 22, no. 3, pp. 963–977, 2018.
- [153] N. C. Oza and S. Russell, *Online ensemble learning*. University of California, Berkeley, 2001.
- [154] A. Fern and R. Givan, "Online ensemble learning: An empirical study," *Machine Learning*, vol. 53, no. 1, pp. 71–109, 2003.
- [155] Y. Sun, K. Tang, L. L. Minku, S. Wang, and X. Yao, "Online ensemble learning of data streams with gradually evolved classes," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 6, pp. 1532–1545, 2016.
- [156] J. Basak, "Online adaptive decision trees," *Neural computation*, vol. 16, no. 9, pp. 1959–1981, 2004.
- [157] F. Leveni, G. W. Cassales, B. Pfahringer, A. Bifet, and G. Boracchi, "Online isolation forest," *arXiv preprint arXiv:2505.09593*, 2025.
- [158] A. Milani, "Online genetic algorithms," 2004.
- [159] S. Mishra, A. Goyal, and D. Sahu, "A comprehensive model leveraging convolutional neural networks for online transaction fraud detection," in *2025 International Conference on Pervasive Computational Technologies (ICPCT)*. IEEE, 2025, pp. 191–197.
- [160] P. Nagarajan and B. Kamali, "Fraud detection in online transaction using hybrid cnn and xgboost," in *2024 International Conference on Emerging Research in Computational Science (ICERCS)*. IEEE, 2024, pp. 1–5.
- [161] B. Qu, Z. Wang, M. Gu, D. Yagi, Y. Zhao, Y. Shan, and F. Zahradnik, "Multi-task cnn behavioral embedding model for transaction fraud detection," in *2024 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 2024, pp. 286–292.
- [162] S. Cao, X. Yang, C. Chen, J. Zhou, X. Li, and Y. Qi, "Titant: Online real-time transaction fraud detection in ant financial," *arXiv preprint arXiv:1906.07407*, 2019.
- [163] H. Xu, S. Li, K. Niu, and G. Ping, "Utilizing deep learning to detect fraud in financial transactions and tax reporting," *Journal of Economic Theory and Business Management*, vol. 1, no. 4, pp. 61–71, 2024.
- [164] J. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P.-E. Portier, L. He-Guelton, and O. Caelen, "Sequence classification for credit-card fraud detection," *Expert systems with applications*, vol. 100, pp. 234–245, 2018.
- [165] I. Benchaji, S. Douzi, B. El Ouahidi, and J. Jaafari, "Enhanced credit card fraud detection based on attention mechanism and lstm deep model," *Journal of Big Data*, vol. 8, no. 1, p. 151, 2021.
- [166] N. Tony, "Real-time payment fraud detection using hybrid deep learning models: Combining rnns with attention mechanisms," *Available at SSRN 5361578*, 2025.
- [167] M.-C. Lee, J.-C. Lin, and E. G. Gran, "Repad: real-time proactive anomaly detection for time series," in *International Conference on Advanced Information Networking and Applications*. Springer, 2020, pp. 1291–1302.
- [168] J. Raval, P. Bhattacharya, N. K. Jadav, S. Tanwar, G. Sharma, P. N. Bokoro, M. Elmorsy, A. Tolba, and M. S. Raboaca, "Raksha: A trusted explainable lstm model to classify fraud patterns on credit card transactions," *Mathematics*, vol. 11, no. 8, p. 1901, 2023.
- [169] T. Deng, S. Bi, and J. Xiao, "Transformer-based financial fraud detection with cloud-optimized real-time streaming," in *Proceedings of the 2024 5th International Conference on Big Data Economy and Information Management*, 2024, pp. 702–707.
- [170] Z. Y. Zhao, Z. Zhu, G. Li, W. Wang, and B. Wang, "Generative pre-training at scale: Transformer-based encoding of transactional behavior for fraud detection," *arXiv preprint arXiv:2312.14406*, 2023.
- [171] C. Yu, Y. Xu, J. Cao, Y. Zhang, Y. Jin, and M. Zhu, "Credit card fraud detection using advanced transformer model," in *2024 IEEE International Conference on Metaverse Computing, Networking, and Applications (MetaCom)*. IEEE, 2024, pp. 343–350.
- [172] P. Dubey, P. Dubey, and P. N. Bokoro, "A unified transformer-bdi architecture for financial fraud detection: Distributed knowledge transfer across diverse datasets," *Forecasting*, vol. 7, no. 2, p. 31, 2025.
- [173] G. Singh, P. Singh, and M. Singh, "Advanced real-time fraud detection using rag-based llms," *arXiv preprint arXiv:2501.15290*, 2025.
- [174] A. Senol, G. Agrawal, and H. Liu, "Joint detection of fraud and concept drift inonline conversations with llm-assisted judgment," *arXiv preprint arXiv:2505.07852*, 2025.
- [175] S. Shuster, E. Zaloof, A. Shabtai, and R. Puzis, "Faa framework: A large language model-based approach for credit card fraud investigations," *arXiv preprint arXiv:2506.11635*, 2025.
- [176] L. Jiang, "Detecting scams using large language models," *arXiv preprint arXiv:2402.03147*, 2024.
- [177] M. Lu, Z. Han, Z. Zhang, Y. Zhao, and Y. Shan, "Graph neural networks in real-time fraud detection with lambda architecture," *arXiv preprint arXiv:2110.04559*, 2021.
- [178] M. Lu, Z. Han, S. X. Rao, Z. Zhang, Y. Zhao, Y. Shan, R. Raghunathan, C. Zhang, and J. Jiang, "Bright-graph neural networks in real-time fraud detection," in *Proceedings of the 31st ACM international conference on information & knowledge management*, 2022, pp. 3342–3351.
- [179] Y. Tian, G. Liu, J. Wang, and M. Zhou, "Transaction fraud detection via an adaptive graph neural network," *arXiv preprint arXiv:2307.05633*, 2023.
- [180] Q. Sha, T. Tang, X. Du, J. Liu, Y. Wang, and Y. Sheng, "Detecting credit card fraud via heterogeneous graph neural networks with graph attention," *arXiv preprint arXiv:2504.08183*, 2025.
- [181] J. Lin, X. Guo, Y. Zhu, S. Mitchell, E. Altman, and J. Shun, "Fraudgt: a simple, effective, and efficient graph transformer for financial fraud detection," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 292–300.
- [182] P. Szekely, C. A. Knoblock, J. Slepicka, A. Philpot, A. Singh, C. Yin, D. Kapoor, P. Natarajan, D. Marcu, K. Knight *et al.*, "Building and using a knowledge graph to combat human trafficking," in *International Semantic Web Conference*. Springer, 2015, pp. 205–221.
- [183] P. Nair, J. Liu, C. Vajiac, A. Olligschlaeger, D. H. Chau, M. Cazzolato, C. Jones, C. Faloutsos, and R. Rabbany, "T-net: Weakly supervised graph learning for combatting human trafficking," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 20, 2024, pp. 22 276–22 284.
- [184] D. Cheng, X. Wang, Y. Zhang, and L. Zhang, "Graph neural network for fraud detection via spatial-temporal attention," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 8, pp. 3800–3813, 2020.
- [185] Y. Duan, G. Zhang, S. Wang, X. Peng, W. Ziqi, J. Mao, H. Wu, X. Jiang, and K. Wang, "Cat-gnn: Enhancing credit card fraud detection via causal temporal graph neural networks," 2024. [Online]. Available: <https://arxiv.org/abs/2402.14708>
- [186] S. Xiang, M. Zhu, D. Cheng, E. Li, R. Zhao, Y. Ouyang, L. Chen, and Y. Zheng, "Semi-supervised credit card fraud detection via attribute-driven graph representation," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, p. 14557–14565, Jun. 2023. [Online]. Available: <http://dx.doi.org/10.1609/aaai.v37i12.26702>
- [187] D. Saldaña-Ulloa, G. De Ita Luna, and J. R. Marcial-Romero, "A temporal graph network algorithm for detecting fraudulent transactions on online payment platforms," *Algorithms*, vol. 17, no. 12, 2024. [Online]. Available: <https://www.mdpi.com/1999-4893/17/12/552>
- [188] C. Ju, X. Ma, and B. Dong, "Real-time cross-border payment fraud detection using temporal graph neural networks: A deep learning approach," *Academic Journal of Sociology and Management*, vol. 3, no. 2, pp. 1–12, Mar. 2025.
- [189] S. Carta, G. Fenu, D. R. Recupero, and R. Saia, "Fraud detection for e-commerce transactions by employing a prudential multiple consensus model," *Journal of Information Security and Applications*, vol. 46, pp. 13–22, 2019.
- [190] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation," 2024. [Online]. Available: <https://arxiv.org/abs/2403.03952>

- [191] L. Song, C. Tekin, and M. Van Der Schaar, "Online learning in large-scale contextual recommender systems," *IEEE Transactions on Services Computing*, vol. 9, no. 3, pp. 433–445, 2014.
- [192] J. Xiao, M. Wang, B. Jiang, and J. Li, "A personalized recommendation system with combinational algorithm for online learning," *Journal of ambient intelligence and humanized computing*, vol. 9, no. 3, pp. 667–677, 2018.
- [193] A. Egg, "Online learning for recommendations at grubhub," in *Proceedings of the 15th ACM Conference on Recommender Systems*, 2021, pp. 569–571.
- [194] C. Constantinides, S. Shiaeles, B. Ghita, and N. Kolokotronis, "A novel online incremental learning intrusion prevention system," in *2019 10th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 2019, pp. 1–6.
- [195] M. Nakıp and E. Gelenbe, "Online self-supervised deep learning for intrusion detection systems," *IEEE Transactions on Information Forensics and Security*, vol. 19, p. 5668–5683, 2024. [Online]. Available: <http://dx.doi.org/10.1109/TIFS.2024.3402148>
- [196] T. Tantisripreecha and N. Soonthomphisaj, "Stock market movement prediction using lda-online learning model," in *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 2018, pp. 135–139.
- [197] T. Singh, R. Kalra, S. Mishra, Satakshi, and M. Kumar, "An efficient real-time stock prediction exploiting incremental learning and deep learning," *Evolving Systems*, vol. 14, no. 6, pp. 919–937, 2023.
- [198] T. Levina and G. Shafer, "Portfolio selection and online learning," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 16, no. 04, pp. 437–473, 2008.
- [199] B. Li and S. C. Hoi, "Online portfolio selection: A survey," *ACM Computing Surveys (CSUR)*, vol. 46, no. 3, pp. 1–36, 2014.
- [200] O. Anava, E. Hazan, S. Mannor, and O. Shamir, "Online learning for time series prediction," in *Conference on learning theory*. PMLR, 2013, pp. 172–184.
- [201] V. Kuznetsov and M. Mohri, "Time series prediction and online learning," in *Conference on Learning Theory*. PMLR, 2016, pp. 1190–1213.
- [202] Z. Shao, S. Yuan, and Y. Wang, "Adaptive online learning for iot botnet detection," *Information Sciences*, vol. 574, pp. 84–95, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025521005697>
- [203] H. Ren, D. Anicic, and T. A. Runkler, "Tinyol: Tinyml with online-learning on microcontrollers," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [204] C. S. Lee and A. Y. Lee, "Clinical applications of continual learning machine learning," *The Lancet Digital Health*, vol. 2, no. 6, pp. e279–e281, 2020.
- [205] F. Amrollahi, S. P. Shashikumar, A. L. Holder, and S. Nemati, "Leveraging clinical data across healthcare institutions for continual learning of predictive risk models," *Scientific reports*, vol. 12, no. 1, p. 8380, 2022.
- [206] A. Singh, M. B. Gurbuz, S. S. Gantha, and P. Jasti, "Class-incremental continual learning for general purpose healthcare models," *arXiv preprint arXiv:2311.04301*, 2023.
- [207] C. Yu, J. Liu, S. Nemati, and G. Yin, "Reinforcement learning in healthcare: A survey," *ACM Computing Surveys (CSUR)*, vol. 55, no. 1, pp. 1–36, 2021.
- [208] Y. Sahin, S. Bulkan, and E. Duman, "A cost-sensitive decision tree approach for fraud detection," *Expert Systems with Applications*, vol. 40, no. 15, pp. 5916–5923, 2013.
- [209] B. Lebichot, G. M. Paldino, W. Siblini, L. He-Guelton, F. Oblé, and G. Bontempi, "Incremental learning strategies for credit cards fraud detection," *International Journal of Data Science and Analytics*, vol. 12, no. 2, pp. 165–174, 2021.
- [210] F. Anowar and S. Sadaoui, "Incremental learning framework for real-world fraud detection environment," *Computational Intelligence*, vol. 37, no. 1, pp. 635–656, 2021.
- [211] N. Innan, A. Sawaika, A. Dhor, S. Dutta, S. Thota, H. Gokal, N. Patel, M. A.-Z. Khan, I. Theodonis, and M. Bennai, "Financial fraud detection using quantum graph neural networks," *Quantum Machine Intelligence*, vol. 6, no. 1, p. 7, 2024.
- [212] A. Ganapathy *et al.*, "Quantum computing in high frequency trading and fraud detection," *Engineering International*, vol. 9, no. 2, pp. 61–72, 2021.
- [213] M. Grossi, N. Ibrahim, V. Radescu, R. Loredi, K. Voigt, C. Von Altrock, and A. Rudnik, "Mixed quantum-classical method for fraud detection with quantum feature selection," *IEEE Transactions on Quantum Engineering*, vol. 3, pp. 1–12, 2022.